

Chapitre 1 : Rappels sur les systèmes d'exploitation

I. Introduction

Un **système d'exploitation (SE)** est un ensemble de programmes qui :

- **Gère les ressources matérielles** (processeur, mémoire, périphériques...),
- **Fait le lien entre les applications et le matériel**, rendant l'ordinateur plus simple à utiliser.

Il peut être vu comme :

1. **Machine virtuelle** : le SE donne à chaque utilisateur l'impression d'avoir son propre ordinateur.
2. **Gestionnaire de ressources** : il partage équitablement et sans conflit les ressources entre les utilisateurs et les programmes.

II. Fonctions d'un système d'exploitation

1. Gestion de la mémoire centrale

- Alloue et libère la mémoire pour les programmes.
- Garde la trace de l'utilisation de la mémoire.
- Utilise la **mémoire virtuelle** pour simuler une grande mémoire à partir des disques secondaires.

2. Gestion des processus et du processeur

- Un **processus** = programme en cours d'exécution.
- Le SE crée, détruit, suspend ou reprend les processus.
- Il décide quel processus utilise le processeur (→ **ordonnancement** ou **scheduling**).

3. Gestion des fichiers

- Fournit une **vue logique unique** des mémoires secondaires (disques, bandes, etc.).
- Crée, supprime et gère les **fichiers** et **répertoires**.
- Permet aux programmes d'accéder aux fichiers via des **primitives** (fonctions du SE).

4. Gestion des entrées/sorties (E/S)

- Cache la complexité du matériel aux utilisateurs.
- Comprend :
 - une **interface commune** pour tous les périphériques,
 - des **pilotes (drivers)** spécifiques à chaque appareil.

5. Protection et sécurité

- Empêche les processus de se perturber entre eux (surtout en **multiutilisateur**).
- Met en place des **mécanismes de sécurité** pour protéger contre les attaques internes et externes.

III. Historique des systèmes d'exploitation

Les systèmes d'exploitation ont évolué **avec les générations d'ordinateurs** et les avancées technologiques.

1. Première génération (1945–1955) — *Sans système d'exploitation*

- Matériel : tubes électroniques, très coûteux, lents et peu fiables.
- Les programmes étaient écrits en **langage machine**, chargés manuellement en mémoire.
- **Un seul programme à la fois** (mode séquentiel).
- Utilisation des **cartes perforées** pour charger les programmes.

2. Deuxième génération (1955–1965) — *Monoprogrammation et traitement par lots*

- Apparition des **transistors** → machines plus fiables.
- Introduction du **traitement par lots (batch processing)** : plusieurs programmes sont préparés, puis exécutés **un à un automatiquement**.
- Langages utilisés : **Fortran** et **Assembleur**.
- Pas encore d'interaction directe avec la machine.

3. Troisième génération (1965–1980) — *Multiprogrammation et temps partagé*

- Utilisation des **circuits intégrés** → meilleure performance.
- **Multiprogrammation** : plusieurs programmes en mémoire, le processeur passe de l'un à l'autre quand un attend une E/S.
- **Spooling (Simultaneous Peripheral Operation On Line)** : gestion automatique des entrées/sorties via disque.
- **Systèmes à temps partagé** : plusieurs utilisateurs connectés simultanément à la même machine via terminaux.

□ 4. Quatrième génération (1980–1990) — *PC, réseaux et temps réel*

- Apparition des **ordinateurs personnels (PC)**.
- **Systèmes d'exploitation en réseau** : permettent le partage de fichiers et la connexion entre ordinateurs.
- **Systèmes temps réel** : utilisés dans les domaines industriels ou critiques où la réaction rapide est essentielle (ex : contrôle industriel, aviation).

5. Cinquième génération (1990–aujourd'hui) — *Multiprocesseurs et systèmes embarqués*

- Objectif : plus de puissance, fiabilité et partage.
- **Systèmes multiprocesseurs** :
 - **Distribués** : plusieurs machines reliées en réseau partagent leurs ressources (ex : campus universitaire).
 - **Parallèles** : plusieurs processeurs dans une seule machine (ex : supercalculateurs).
- **Systèmes embarqués** : systèmes intégrés dans des appareils électroniques (téléphones, TV, voitures, etc.).
 - Exemple : **Android**.

- Ils sont souvent autonomes ou connectés (Bluetooth, Wi-Fi, etc.).

IV. Historique d'UNIX et de LINUX

- **UNIX** : créé dans les années **1960** aux **Bell Labs** par **Ken Thompson** et **Dennis Ritchie**.
 - Réécrit en **langage C (1973)** → devient **portable** sur plusieurs machines.
 - Commercialisé dès **1975**.
 - Standardisé par la norme **POSIX** pour éviter les incompatibilités.
- **LINUX** : créé en **1991** par **Linus Torvalds**, inspiré d'UNIX.
 - Version **libre et open source** d'UNIX.
 - Diffusé sous différentes **distributions** : Ubuntu, Debian, Fedora, Red Hat, etc.
 - Très populaire sur les **serveurs** et chez les **particuliers**.

V. Caractéristiques d'UNIX / LINUX

- **Multiutilisateur et multitâche**.
- **Système hiérarchique de processus** : chaque processus hérite de caractéristiques de son parent.
- **Création dynamique** de processus avec **fork()**.
- **Communication entre processus** : sémaphores, tubes, signaux, sockets...
- **Threads** : plusieurs processus peuvent partager le même programme en mémoire.
- **Système de fichiers hiérarchique** : tout est organisé en répertoires.
- **Entrées/Sorties intégrées aux fichiers** : chaque périphérique est vu comme un fichier.
- **Mémoire virtuelle** : échange entre RAM et disque dur pour optimiser les performances.
- **Protection et sécurité** : mot de passe, droits sur les fichiers, super-utilisateur (**root**).

VI. Interaction Utilisateur – Système d'exploitation

- Le SE fournit aux programmes des **bibliothèques d'appels système** (comme en C sous UNIX).
- L'utilisateur interagit via un **interpréteur de commandes (Shell)** :
 - Interface entre l'utilisateur et le système.
 - Permet de lancer des programmes, gérer des fichiers, configurer le système, etc.
 - Contient un **langage de scripts** pour automatiser les tâches.

VII. Interface Système d'exploitation – Matériel

Le SE gère les composants matériels grâce à des **signaux appelés interruptions** :

- **Interruptions matérielles** : générées par le matériel (ex : horloge pour gérer le temps processeur).
- **Interruptions logicielles** : générées par les programmes (ex : division par zéro, défaut de page, appel système).

Ces interruptions permettent au SE de **reprendre le contrôle du système** à tout moment.

VII. Appels système

Les **processeurs** fonctionnent sous deux **modes d'exécution** :

1. **Mode superviseur (noyau) :**
 - Réservé au **système d'exploitation**.
 - Toutes les instructions sont autorisées.
2. **Mode utilisateur :**
 - Pour les **programmes des utilisateurs**.
 - Certaines instructions sont interdites pour éviter les erreurs ou intrusions.

Ce **double mode** protège le système d'exploitation.

Définition d'un appel système

Un **appel système** est une **interruption logicielle** qui permet à un programme utilisateur d'accéder à un service du système d'exploitation.

Le déroulement :

1. Passage du **mode utilisateur** au **mode superviseur**.
2. Lecture et vérification des **paramètres**.
3. Exécution de la **fonction demandée**.
4. Retour au **programme utilisateur**.

Catégories d'appels système

1. **Gestion de processus** : créer, exécuter, terminer, suspendre un processus, attendre un signal, etc.
2. **Manipulation de fichiers et périphériques** : ouvrir, fermer, lire, écrire, créer, supprimer, etc.
3. **Maintenance et information** : obtenir ou modifier l'heure, la date, etc.

VII.1 Implémentation des appels système

Les paramètres d'un appel système peuvent être transmis de plusieurs façons :

- En copiant le **numéro de fonction** dans un registre.
- En mettant les **paramètres** dans des registres ou en mémoire.
- Le système exécute ensuite la fonction correspondante.

VII.2 Programmes système

Ce sont les **commandes et outils fournis par le système d'exploitation**.

Catégories principales :

- **Manipulation de fichiers** : créer, copier, supprimer, imprimer...
- **Maintenance d'informations** : date, heure, etc.
- **Édition / modification** : éditeur de texte, etc.
- **Support de langages** : compilateurs, interpréteurs.
- **Applications système** : tableurs, éditeurs, etc.

- **Applications utilisateurs** : programmes créés par les utilisateurs.

VIII. Types de noyaux des systèmes d'exploitation

Les **noyaux** (kernels) gèrent les ressources et la communication entre matériel et logiciel.

Deux grandes **architectures** existent :

VIII.1 Noyau monolithique

- Tous les services (gestion de mémoire, fichiers, pilotes, réseau...) sont intégrés dans un **seul bloc** au **mode superviseur**.
- Les applications tournent en **mode utilisateur**.
- Avantage : **rapide**, car communication directe entre services.
- Inconvénient : **moins sécurisé** (tous les services partagent le même espace mémoire).
- Exemple : **UNIX**.

VIII.2 Micro-noyau

- Structure **moderne** et **sécurisée**.
- Le noyau ne gère que les **fonctions essentielles** : allocation du processeur et de la mémoire.
- Les autres services (fichiers, réseau, pilotes, etc.) sont séparés et tournent en **mode utilisateur**.
- Avantage : **meilleure sécurité** et stabilité.
- Inconvénient : **moins performant** à cause des échanges entre services.
- Exemple : **RTLinux** (version temps réel).

Différence principale :

Architecture	Services en mode superviseur	Services en mode utilisateur	Exemple
Monolithique	Tous	Applications uniquement	UNIX
Micro-noyau	Fonctions de base (CPU, mémoire)	Système de fichiers, pilotes, etc.	RTLinux

IX. Implémentation des systèmes d'exploitation

- À l'origine, les **SE** étaient écrits en langage **Assembleur** (ex : **MS-DOS**).
- Aujourd'hui, ils sont majoritairement écrits en langage de haut niveau (**C**) :
 - **UNIX / Linux** → C
 - **Windows NT** → C

Avantages du C par rapport à l'Assembleur

- Code **plus compact** et **modulaire**.
- **Plus facile à déboguer**.
- **Portable** (peut fonctionner sur plusieurs machines).

Remarque : l'assembleur reste plus **rapide et optimisé**, mais beaucoup moins **portable** et **complexe à maintenir**.

Partie II : Système de Gestion de Fichiers (SGF)

I. Introduction

Le **Système de Gestion de Fichiers (SGF)** permet de **stocker, organiser et accéder efficacement aux données** sur un disque.

♦ Fichier

- Un **fichier** est une **unité logique de stockage** contenant des informations enregistrées de manière permanente.
- Chaque fichier a :
 - un **nom**,
 - un **emplacement** sur le disque,
 - et des **attributs** (taille, propriétaire, permissions...).

♦ Répertoire (ou dossier)

- Sert à **organiser les fichiers** pour un accès plus rapide.
- C'est un **fichier spécial** contenant des **entrées** qui associent les noms de fichiers à leurs emplacements.
- Organisation **hiérarchique** (ex : /home/user/docs).

♦ Système de fichiers (File System)

- Partie du **système d'exploitation** qui gère les fichiers : **création, suppression, lecture, écriture, partage et protection**.

II. Catégories de fichiers

1. **Fichiers ordinaires** → contiennent les données de l'utilisateur.
2. **Fichiers répertoires** → regroupent d'autres fichiers/répertoires.
3. **Fichiers spéciaux** → représentent des périphériques (ex : **/dev/sda**, imprimante, etc.).

III. Organisation de l'espace disque

- Les données sont stockées sur des **blocs (ou clusters)** de taille fixe (512, 1024, 2048 octets...).
- Chaque fichier occupe un **ensemble de blocs** numérotés.
- Lecture/écriture = transfert du bloc entier vers la mémoire.

IV. Techniques d'allocation des blocs

Deux principales méthodes :

1. Allocation contiguë

- Le fichier est enregistré dans des **blocs successifs** sur le disque.
Avantage : accès rapide.
Inconvénient : nécessite de connaître la taille du fichier à l'avance, risque de **fragmentation**.

2. Allocation chaînée (non contiguë)

Le fichier est une **liste chaînée de blocs** dispersés sur le disque.

a) Liste chaînée de blocs

- Chaque bloc contient les **données + un pointeur** vers le bloc suivant.
Plus flexible, pas de fragmentation.
Si un lien est perdu → le reste du fichier devient inaccessible.

b) Liste chaînée indexée (FAT)

- Les pointeurs sont stockés dans une **table en mémoire : la FAT (File Allocation Table)**.
- Chaque entrée de la FAT indique le **bloc suivant**.
Accès plus rapide et centralisé.

c) Structure d'I-Node (utilisée sous UNIX/Linux)

- Chaque fichier a un **I-Node** contenant :
 - ses attributs,
 - et les **adresses des blocs** du fichier.
- Le répertoire pointe vers le **I-Node** du fichier.

V. Protection des fichiers

Objectif : **empêcher les accès non autorisés**.

Système de contrôle d'accès :

Chaque fichier a une **liste d'accès (ACL)** indiquant :

- les **utilisateurs** autorisés,
- et les **droits** (lecture, écriture, exécution).

Pour simplifier, les utilisateurs sont regroupés en trois catégories :

1. **USER (propriétaire)**
2. **GROUP (groupe d'utilisateurs)**
3. **OTHER (tous les autres)**

Exemple UNIX :

`-rwxr-xr--`

- `rwx` → lecture, écriture, exécution pour **USER**
- `r-x` → lecture, exécution pour **GROUP**
- `r--` → lecture seulement pour **OTHER**

VI. Partage de fichiers

Les fichiers peuvent être **partagés** via des **liens**.

1. Lien physique (hard link)

- C'est un **autre nom** pointant vers le **même I-Node**.
- Créé avec : `ln fichier_original nouveau_nom`
- Le fichier n'est supprimé que lorsque **tous les liens physiques** sont effacés.

2. Lien symbolique (symlink)

- C'est un **fichier spécial** contenant le **chemin du fichier cible**.
- Créé avec : `ln -s fichier_original lien_symbolique`
- Il a un **I-Node différent**.
- Si le fichier d'origine est supprimé, le lien symbolique devient **cassé** (dangling link).

Exemple :

- 1) Créer un fichier fic1
- 2) Indiquer son numéro d'i-node .
- 2) Créer un lien physique fic2 sur fic1.
- 3) Indiquer son numéro d'i-node et conclure sur fic1 et fic2.
- 4) Créer un lien symbolique fic3 sur fic1.
- 5) Indiquer son numéro d'i-node et conclure sur fic1 et fic3.
- 6) Modifier le contenu de fic2 et visualiser alors le contenu de fic1.
- 7) Modifier le contenu de fic3 et visualiser alors le contenu de fic1.
- 8) On supprime le fichier fic1 et conclure sur fic2 et fic3.

Chapitre 2 – Système de Gestion de Fichiers (SGF)

Le **Système de Gestion de Fichiers (SGF)** est une partie essentielle du **système d'exploitation (SE)**.

Il a pour but d'assurer un **accès simple, structuré et efficace aux données stockées sur les supports physiques** (disque dur, clé USB, CD, etc.).

1. Concept de base

1.1. Le concept de fichier

- Un **fichier** est une **unité logique de stockage** utilisée pour conserver des données de manière permanente.
- Il peut contenir du **texte, des images, des programmes, des calculs, etc.**
- Les fichiers sont en général créés par les utilisateurs, mais certains sont générés automatiquement par le système ou des outils (ex : compilateurs).

Le **système d'exploitation** établit la **correspondance entre le fichier logique et les données binaires** stockées physiquement, sans que l'utilisateur ait à s'en soucier.

Fondamental : les attributs d'un fichier

Chaque fichier possède des **attributs** (ou propriétés) qui permettent de l'identifier et de le décrire :

- **Nom**
- **Extension** (type du fichier, ex : **.txt**, **.exe**, **.jpg**...)
- **Date et heure** de création ou de dernière modification
- **Taille** (en octets)
- **Droits d'accès / protection** (lecture, écriture, exécution)

Certains attributs sont définis par l'utilisateur, d'autres par le système d'exploitation.

Les opérations sur les fichiers

Le système d'exploitation fournit un ensemble d'opérations pour manipuler les fichiers :

Opération	Description
Création	Créer un nouveau fichier vide
Écriture	Insérer ou ajouter des données (à partir d'un pointeur d'écriture)
Lecture	Lire le contenu du fichier (à partir d'un pointeur de lecture)
Positionnement	Se déplacer à une position donnée dans le fichier
Suppression	Supprimer le fichier et libérer son espace
Troncature	Remettre la taille à zéro tout en gardant les attributs

Ajout / concaténation	Ajouter du contenu à la fin d'un fichier
Renommage	Changer le nom du fichier
Copie	Créer un duplicata du fichier ou un nouveau nom pointant vers le même contenu
Ouverture	Associer un fichier à un processus pour permettre des opérations
Fermeture	Libérer les ressources liées au fichier après utilisation

Exemple :

\$ cat > fichier

abcd

→ crée un fichier nommé *fichier* contenant le texte "abcd".

1.2. Le système de gestion de fichiers (SGF)

Définition

Le **SGF** est le **module du système d'exploitation** chargé de **gérer le stockage, l'accès et la manipulation des fichiers** sur les supports physiques.

Il agit comme une **interface** entre :

- L'utilisateur et ses données
- Le matériel de stockage (disque, clé USB, CD...)

Rôles et fonctions principales

Le SGF permet :

1. **D'entreposer** les données sous forme de fichiers sur un ou plusieurs périphériques de stockage.
2. **D'organiser** les informations de façon hiérarchique (répertoires, dossiers, sous-dossiers).
3. **De simplifier l'accès** aux périphériques pour l'utilisateur et le programmeur.
4. **D'assurer la transparence** vis-à-vis :
 - du **type de périphérique** (disque dur, clé USB, etc.)
 - du **format ou de l'organisation interne** du support (FAT, NTFS, ext4, etc.)



Illustration d'un système de gestion de fichiers

1.3 – Notion de répertoire

Définition

Un **répertoire** (ou **dossier**, **directory**, **catalogue**) est une **structure utilisée pour organiser les fichiers** sur un support de stockage (disque dur, clé USB, etc.).

Il permet de **classer et retrouver facilement les fichiers**, surtout lorsqu'ils sont très nombreux. En réalité, **un répertoire est lui-même un fichier spécial**, créé par le **Système de Gestion de Fichiers (SGF)**, dont le contenu sert à **référencer d'autres fichiers ou sous-répertoires**.

Complément (vue SGF)

Du point de vue du **système de gestion de fichiers** :

- Un **répertoire** est un **fichier structuré logiquement comme un tableau**.
- **Chaque ligne (entrée)** du tableau correspond à **un fichier** contenu dans le répertoire.
- Cette **entrée** associe le **nom du fichier** à ses **informations internes** (appelées attributs).
- L'entrée peut :
 - Contenir directement ces **attributs** (taille, date, adresse, etc.), ou
 - **Pointer vers une autre structure** où ces informations sont stockées.

Exemple :

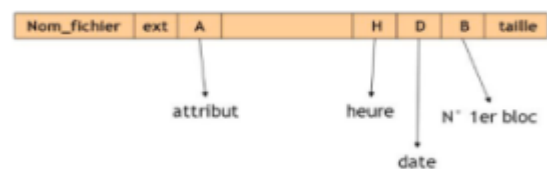
Chaque **entrée de répertoire** contient les **informations essentielles** sur un fichier, notamment :

- le **nom du fichier**,
- son **extension**,
- un **attribut** indiquant s'il s'agit d'un **fichier** ou d'un **répertoire**,
- la **date et l'heure** de création,
- le **numéro du premier bloc** occupé sur le disque,
- et la **taille** du fichier.

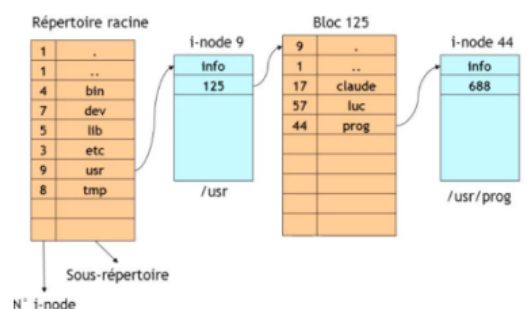
Ces informations permettent au système d'exploitation de **localiser et gérer efficacement** chaque fichier.

Chemin d'un fichier

- Le **nom complet d'un fichier** (ou **chemin**) indique son **emplacement dans la hiérarchie** des répertoires, depuis le **répertoire racine** jusqu'au fichier.
- Les **séparateurs de répertoires** varient selon le système d'exploitation :
 - / → **UNIX/Linux**
 - \ → **Windows/DOS**



Structure d'un répertoire : cas de MSDOS(32 octets)



Structure d'un répertoire : cas d'UNIX (14 octets)

- $\rightarrow$ **Multics**
- $\rightarrow$ **MacOS**
- Un **chemin absolu** commence à partir de la racine (ex : `/home/user1/rapport.txt`), tandis qu'un **chemin relatif** part du répertoire courant.
- Le système d'exploitation distingue un **fichier** d'un **répertoire** grâce à un **bit d'attribut spécifique**.

2. Rôles d'un Système de Gestion de Fichiers (SGF)

Le **SGF** a pour mission principale de **gérer les fichiers** et de **fournir les outils nécessaires à leur manipulation**.

Ses rôles essentiels sont :

- Offrir une **interface simple et conviviale** pour l'utilisateur (création, ouverture, copie, suppression, renommage, etc.).
- **Gérer l'organisation des fichiers** sur le disque (allocation de l'espace).
- **Suivre l'espace libre** sur le disque dur.
- **Gérer les accès multiples** en environnement multi-utilisateurs.
- Fournir des **outils de diagnostic** et de **récupération** en cas d'erreurs.

3. Gestion de l'organisation de l'espace disque

Les données sont stockées en **blocs** sur le disque.

Pour simplifier la gestion, les blocs sont regroupés en **clusters** (unités d'allocation).

- Tous les clusters ont la **même taille**.
- La **taille minimale** d'un cluster est **1 bloc**.
- La taille du cluster influence les performances et le gaspillage d'espace.

3.1. Compromis sur la taille des clusters

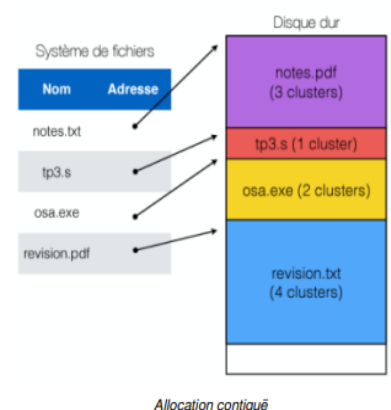
- **Grands clusters** $\rightarrow$ Moins de gestion, mais **plus d'espace gaspillé**.
Exemple : pour un fichier de 1000 octets
 - Cluster de 512 o $\rightarrow$ 24 o gaspillés
 - Cluster de 2048 o $\rightarrow$ 1048 o gaspillés
- **Petits clusters** $\rightarrow$ Moins de gaspillage, mais **plus difficile à gérer et accès plus lents** si les blocs ne sont pas contigus.

3.2. Techniques d'allocation des blocs

Trois méthodes principales d'organisation des blocs sur le disque :

3.2.1. Allocation contiguë $\rightarrow$ Les blocs du fichier sont placés à la suite les uns des autres.

- Avantage : accès rapide (les blocs sont voisins).
- Inconvénient : difficulté à prévoir la taille du fichier et risque de fragmentation.



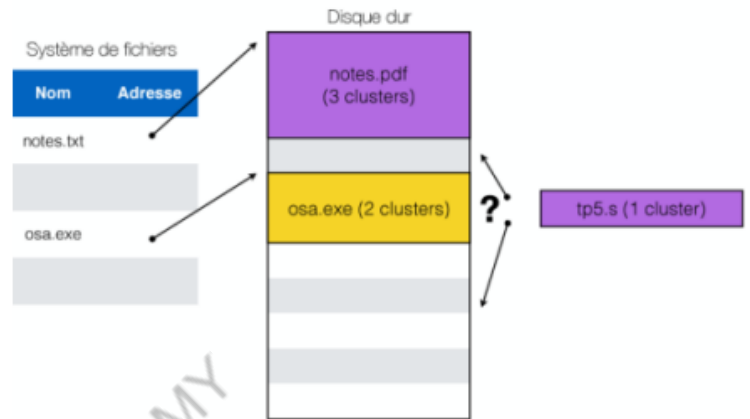
- Utilise une **table d'emplacement** indiquant le point de départ de chaque fichier sur le disque.

Avantage :

- Très **rapide** en lecture/écriture, car les **blocs du fichier sont contigus** sur le disque.
- Moins de déplacements de la tête de lecture → **meilleure performance**.

Inconvénients :

- **Gaspillage d'espace** : le dernier bloc est souvent partiellement vide.
- **Difficile d'anticiper la taille** du fichier → il faut réserver trop ou pas assez de blocs.
- Si le fichier s'agrandit, il faut le **déplacer**, ce qui ralentit le système.
- **Fragmentation externe** : de petits espaces libres inutilisables apparaissent au fil du temps.

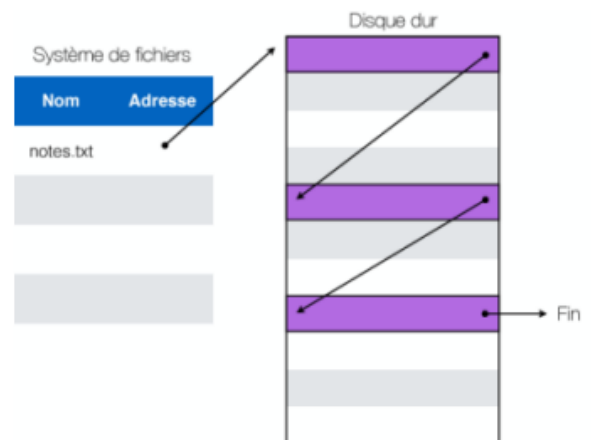


3.2.2.Allocation chaînée (non contiguë) —

Allocation contiguë - cas de la fragmentation externe

Principe :

- Les **blocs d'un fichier sont reliés entre eux** par des pointeurs.
- Le fichier peut être **dispersé sur le disque**.
- Lorsqu'il change de taille, il suffit d'ajouter ou de retirer des blocs.
- **Aucune limite de taille**, sauf celle de l'espace disque disponible.



Avantages :

- Évite la **fragmentation externe**.
- **Simple à mettre en œuvre**, sans structure complexe.

Inconvénients :

- **Accès uniquement séquentiel**, donc lent pour un accès direct.
- **Risque de perte totale** du fichier si un chaînage est corrompu.
- Une **erreur de pointeur** peut rediriger vers une mauvaise zone mémoire.

3.2.3.Allocation non contiguë indexée

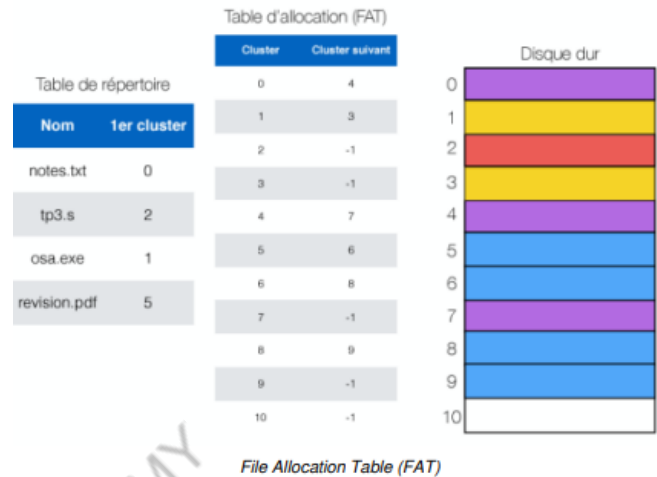
Ce mode d'allocation corrige les limites de l'allocation chaînée en **stockant les pointeurs des blocs dans une table centrale** plutôt que dans les blocs eux-mêmes.

Principe :

- Les informations sur les blocs sont conservées dans une **structure en mémoire** (table).
- Chaque système utilise une méthode spécifique :
 - **MS-DOS** → utilise la **FAT (File Allocation Table)**
 - **Windows NT** → utilise la **MFT (Master File Table)** du système NTFS
 - **UNIX / Linux** → utilisent les **I-nodes**

a/FAT (File Allocation Table) :

- Contient **deux tables** :
 1. **Table de répertoire** → nom, chemin et premier cluster de chaque fichier.
 2. **Table d'allocation (FAT)** → décrit l'utilisation de chaque cluster.
- Chaque fichier est une **chaîne de clusters** : chaque cluster pointe vers le suivant.
- '0' indique un cluster libre, et '-1' marque la fin d'un fichier.



AT16 :

- Utilisé par **MS-DOS**.
- Les numéros de blocs sont codés sur **16 bits**.
- Taille maximale adressable : **2 Go** ($2^{16} \times 32$ Ko).

FAT32 :

- Utilisé à partir de **Windows 95**.
- Numéros de blocs sur **32 bits** (en réalité 28 bits, 4 réservés).
- Taille maximale théorique : **8 To**, mais **limitée à 32 Go** par Microsoft pour encourager l'usage de NTFS.

b/NTFS (New Technology File System) :

- Utilisé par **Windows NT, 2000, XP, Vista, etc.**
- Basé sur une structure appelée **MFT (Master File Table)**, contenant des informations détaillées sur les fichiers.
- Permet les **noms longs** et est **sensible à la casse** (différencie majuscules/minuscules).
- **Performances supérieures** grâce à un **arbre binaire** pour localiser rapidement les fichiers.
- Offre une **meilleure sécurité** en permettant d'attribuer des **droits et attributs spécifiques** à chaque fichier.

c/Structure d'un I-Node (Index Node)

Définition :

Un **inode** est une structure utilisée par les systèmes de fichiers **UNIX/Linux (ext3fs)** pour stocker les **informations sur un fichier**.

Il contient **10 pointeurs directs** vers des blocs de données sur le disque.

Structure d'un inode :

- **Attributs du fichier :**
 - Taille du fichier
 - Identité du **propriétaire** et du **groupe**
 - **Droits d'accès** (lecture **r**, écriture **w**, exécution **x**) pour :
 - le **propriétaire** (user)
 - le **groupe** (group)
 - les **autres** (others)
 - **Dates** : création, dernière lecture, dernière modification
 - **Nombre de liens** (références vers le fichier)
 - **Adresses des blocs de données**

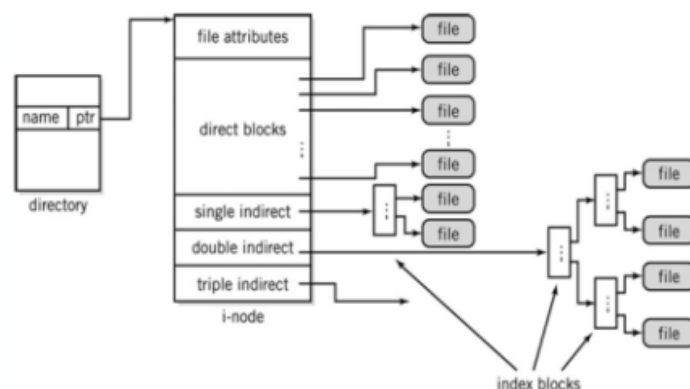
Exemple de permissions :

`-rwxr-xr-- ubuntu ubuntu fichier`

- **rwx** → lecture, écriture, exécution pour le **user**
- **r-x** → lecture et exécution pour le **group**
- **r--** → lecture seule pour les **autres**

Types d'indirections (pour fichiers volumineux) :

- **Simple indirection** → un pointeur vers un bloc contenant plusieurs adresses de blocs de données.
- **Double indirection** → un pointeur vers un bloc d'index contenant d'autres blocs d'index menant aux blocs de données.
- **Triple indirection** → même principe mais sur **trois niveaux**, pour gérer des fichiers très grands.

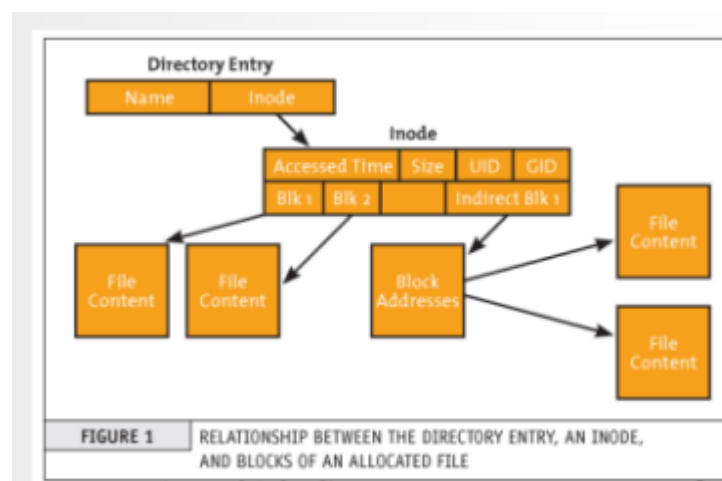


Structure d'un Inode

La relation entre l'entrée du répertoire, l'inode et les blocs de données est illustré dans la figure suivante :

Complément sur les I-Node et la gestion de l'espace disque

Rôle du I-Node :



- Le **but** de la structure I-Node est **d'alléger les répertoires**.
- Les répertoires ne contiennent **que les noms** des fichiers/sous-répertoires **et le numéro d'I-Node associé**.
- Toutes les informations (attributs, emplacement des blocs, etc.) sont stockées dans le **I-Node** lui-même.

Exemple (taille maximale d'un fichier sous UNIX) :

Si un bloc = **1 Ko**, la taille max théorique d'un fichier est :

10 × 1 Ko (pointeurs directs)

- 256 × 1 Ko (simple indirection)
 - 256² × 1 Ko (double indirection)
 - 256³ × 1 Ko (triple indirection)
- ≈ **16 Go** (en théorie)

En pratique, la taille réelle est un peu inférieure à cause des **pointeurs signés**, mais **3 accès disque** suffisent pour atteindre n'importe quel octet du fichier.

3.3. La Gestion de l'espace libre sur le disque :

Les systèmes de fichiers doivent suivre quels blocs sont **libres ou utilisés**.

Deux principales méthodes :

1. Bitmap (Unix, NTFS)

- Une suite de bits représente l'état de chaque cluster.
- 0 → libre ; 1 → occupé.
- Exemple : 1000100010001111b → les clusters 0, 4, 8, 12, 13, 14, 15 sont utilisés.

2. Liste doublement chaînée (FAT)

- Les clusters libres sont reliés entre eux dans une liste.
- Lorsqu'un bloc est utilisé, il est **retiré** de la liste.
- Lorsqu'un fichier est **supprimé**, ses blocs sont **remis** dans la liste.

4. Autres systèmes de fichiers et comparaison :

De nombreux systèmes existent (voir Wikipedia : *Comparison of file systems*).

Les **critères de comparaison** incluent :

- Taille max d'un **disque** et d'un **fichier**
- Longueur et caractères autorisés dans les **noms de fichiers**
- **Vitesse d'accès** aux fichiers et aux répertoires
- **Fragmentation** et performance d'accès
- **Tolérance aux erreurs** (ex. : journalisation des transactions)
- **Sécurité** et gestion des **droits d'accès**

5. Étude de cas : Systèmes de fichiers Linux

Principe général :

Sous **Linux**, **tout est fichier** : programmes, périphériques, répertoires, etc.

L'ensemble du système est organisé dans **une seule arborescence**, dont la **racine** est **/**.

L'utilisateur **root** (administrateur) en est le propriétaire principal.

5.1. Les différentes Catégories principales de fichiers :

1. Fichiers ordinaires (-)

- Contiennent des **données** : texte, code, images, sons, vidéos, exécutables, etc.
- Exemples :
 - `.txt`, `.log` → texte
 - `/bin/ls`, `a.out` → exécutables
 - `.jpg`, `.mp3` → multimédias

Exemple de permissions :

`-rwxrw-r-- 1 etudiant 2LR 34568 avril 3 14:34 mon-fichier`

- → C'est un fichier ordinaire.

2. Répertoires (d)

- Servent à **organiser** les fichiers et autres répertoires.
- Chaque répertoire contient des **entrées pointant** vers des fichiers ou sous-dossiers.
- Exemples : `/home`, `/etc`, `/var`.

Exemple de permissions :

`drwxr-x--- 1 etudiant 2LR 13242 avril 2 13:14 mon-repertoire`

- → C'est un répertoire.

<code>/bin</code>	Commandes binaires utilisateur essentielles (pour tous les utilisateurs)
<code>/boot</code>	Fichiers statiques du chargeur de lancement
<code>/dev</code>	Fichiers de périphériques
<code>/etc</code>	Configuration système spécifique à la machine
<code>/home</code>	Répertoires personnels des utilisateurs
<code>/lib</code>	Bibliothèques partagées essentielles et modules du noyau
<code>/mnt</code>	Point de montage pour les systèmes de fichiers montés temporairement
<code>/proc</code>	Système de fichiers virtuel d'information du noyau et des processus
<code>/root</code>	Répertoire personnel de root (optionnel)
<code>/sbin</code>	Binaires système (binaires auparavant mis dans <code>/etc</code>)
<code>/sys</code>	État des périphériques (model device) et sous-systèmes (subsystems)
<code>/tmp</code>	Fichiers temporaires

Le répertoire racine (le répertoire `/`) sous LINUX.

Fichiers spéciaux

Ces fichiers servent à **interagir avec le matériel** (périphériques) ou effectuer certaines **tâches système**.

Ils se trouvent principalement dans `/dev` et sont de deux types :

● a. Fichiers en mode bloc (Block Devices)

- Accès **par blocs** (lecture/écriture par segments).
- Utilisés pour les **disques durs**, **clés USB**, etc.
- Exemples : `/dev/sda`, `/dev/sdb`.

- **b. Fichiers en mode caractère (Character Devices)**
 - Accès **caractère par caractère** (flux continu).
 - Utilisés pour **terminaux, ports série**, etc.
 - Exemples : `/dev/tty`, `/dev/null`.

Fichiers liens

Les **liens** sont des fichiers qui **réfèrent un autre fichier** sans le dupliquer.

Ils permettent d'accéder au même contenu via plusieurs noms.

Exemple :

```
lrwxrwxrwx 1 root root 14 Aug 1 01:58 Mail -> ../../bin/mail*
```

Ici, `Mail` est un **lien symbolique** vers `../../bin/mail`.

Commandes utiles :

- `ls -l` → affiche les permissions et indique le type (`-`, `d`, `l`).
- `file <nom_fichier>` → indique le **type exact** d'un fichier.

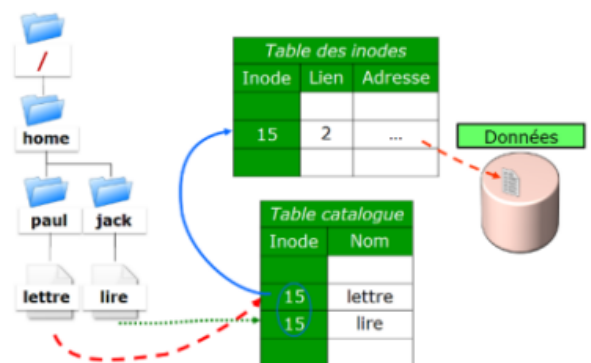
5.2.Partage de fichiers

Un **fichier partagé** apparaît dans plusieurs répertoires ou sous plusieurs noms.

Ce partage se fait grâce aux **liens**, évitant les duplications et maintenant la **cohérence** des données.

Deux types de liens existent :

- **a. Lien physique (hard link)**
 - Fait référence **directement à l'inode** du fichier.
 - Tous les liens pointent vers **les mêmes données**.
 - Si un lien est supprimé, le fichier reste accessible tant qu'il reste au moins **un lien actif**.



Création :

```
$ ln fichier_original lien_physique
```

Suppression :

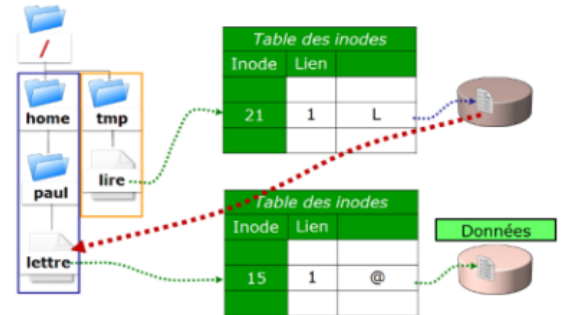
```
$ rm fichier
```

- → Le fichier n'est effacé que lorsque **tous les liens physiques** sont supprimés.

Lien symbolique :

Un **lien symbolique** (ou *symlink*) est un **fichier spécial** qui contient le **chemin d'accès** vers un autre fichier.

- Il se crée avec la commande : `ln -s`.
- Il est identifiable avec `ls -F` (affiché avec un `@` à la fin).
- Contrairement au lien physique, il **n'utilise pas le même inode** que le fichier original.
- Il agit comme un **alias** du fichier cible.
- Si le **fichier d'origine est supprimé**, le lien devient **cassé** (*broken link*).



Différence lien hard vs lien symbolique :

- **Lien hard (physique)** : pas d'inode propre, partage l'inode du fichier cible, limité à la même partition.
- **Lien symbolique** : possède **sa propre inode**, peut pointer vers des fichiers sur **différentes partitions** grâce à l'usage d'un chemin d'accès.

5.3.Implémentation pratique des SGF sous Linux (Ext4 et montage) :

- **Objectif** : Comprendre la structuration du disque par Ext4 et l'intégration des partitions dans l'arborescence Linux via le VFS.

5.3.1.Du Brut au Fichiers : Le Formatage : Transforme une partition vide en système de fichiers utilisable.

- **Choix des SGF Linux** :
 - **Ext4** : Système standard actuel, succède à Ext3 et Ext2, utilise les **extents** pour gérer les gros fichiers et réduire la fragmentation, supporte des partitions très grandes (jusqu'à 1 Exaoctet).
 - **Alternatives** : XFS (grands fichiers), Btrfs (fonctionnalités avancées).
- **Commande mkfs** : Initialise la structure d'un système de fichiers sur une partition (ex. `/dev/sdaX`).

Opération	Rôle	Commande Type
Créer la structure	Écrit le Superbloc, les Inodes et les Bitmaps.	<code>mkfs.ext4 /dev/sda1</code>
Ajuster la taille des Blocs	Choisir l'unité d'allocation (optimale : 4 Ko pour l'alignement mémoire).	<code>mkfs.ext4 -b 4096 /dev/sda1</code>
Ajuster la Densité des Inodes	Définir la limite maximale du nombre de fichiers. Crucial pour les partitions destinées aux petits fichiers.	<code>mkfs.ext4 -i 8192 /dev/sda1</code>

Structure Ext4 après formatage :

- La partition est divisée en **groupes de blocs** pour optimiser les accès disque.
- Chaque groupe contient :
 - **Superbloc** (avec copies de secours)
 - **Bitmaps** pour les inodes et les blocs libres/occupés
 - **Table d'inodes** correspondant au groupe

5.3.2. l'Intégration : Le Processus de Montage

- **VFS (Virtual File System)** : couche d'abstraction qui présente une arborescence unique (/), indépendamment de la provenance des données.
- **Montage (mount)** : attache un système de fichiers d'une partition à un répertoire existant (point de montage).
 - Exemple : `/dev/sda2` monté sur `/home` → `/home` est géré par le SGF de `/dev/sda2`.
- **Automatisation** : le fichier `/etc/fstab` contient la liste des partitions et options de montage pour un montage automatique au démarrage.

5.3.3. Administration et Diagnostics : surveillance de l'espace disque et des inodes pour gérer les fichiers et la santé du SGF.

A. Surveillance de l'Espace et des Inodes :

Commande	Rôle	Exemple d'Usage
<code>df -h</code>	Affiche l'espace disque libre/utilisé de chaque partition.	Diagnostic si une partition manque d'espace.
<code>df -i</code>	Affiche le nombre d' Inodes libres/utilisés sur chaque partition.	Diagnostic si le système ne peut plus créer de fichiers (Inodes épuisés).
<code>du -sh /chemin/</code>	Calcule la taille totale occupée par un répertoire.	Diagnostic pour localiser le répertoire qui consomme le plus d'espace.
<code>ls -li</code>	Affiche le numéro d'Inode d'un fichier.	Vérifier si deux noms différents <u>pointent</u> vers le même Inode (Lien Physique).

Résilience et Entretien des SGF sous Linux :

- **fsck (File System Check)** : vérifie et répare la cohérence du système de fichiers (superblocs, bitmaps, inodes). À exécuter sur une partition démontée ou en lecture seule.
- **tune2fs** : permet de consulter et modifier les paramètres avancés d'Ext4, comme la fréquence des vérifications et le mode de journalisation.

Chapitre 3 : Gestion des processus

1. Rappels

Algorithme : Description abstraite et générale d'un comportement pour résoudre une classe de problèmes. Il doit être général et toujours se terminer.

Programme : Traduction formelle d'un algorithme dans un langage de programmation, destinée à être exécutée par un processeur spécifique.

Processeur : Dispositif physique capable d'interpréter et d'exécuter un programme.

Ressource : Élément nécessaire à l'exécution d'un processus, par exemple : CPU, mémoire centrale, mémoire auxiliaire, périphériques E/S, fichiers, programmes utilitaires, liaisons à d'autres systèmes, etc.

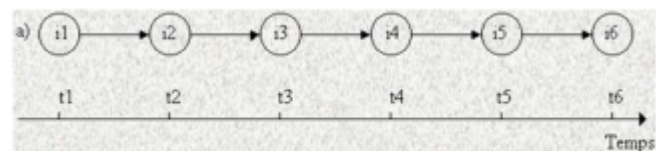
Job / Travail : Demande d'exécution formalisée via le JCL (Job Control Language), composée d'étapes correspondant à des processus. Chaque job est identifié et rattaché à un utilisateur connu du système.

2.1. Introduction

- L'exécution des instructions d'un programme suit un ordre précis de précedence, représentable par un graphe de précedence.
- **Exemple** : Programme lisant deux variables **a** et **b** depuis un fichier, calculant leur somme et affichant les valeurs :
 - i1 : lire a
 - i2 : écrire a
 - i3 : lire b
 - i4 : écrire b
 - i5 : calculer $c = a + b$
 - i6 : écrire c

1. Exécution séquentielle : $i1 < i2 < i3 < i4 < i5 < i6$

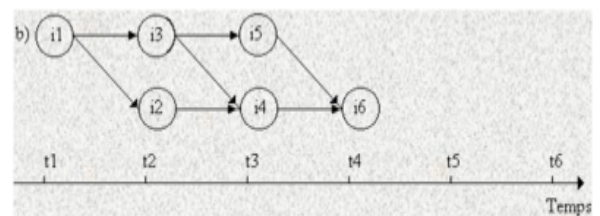
- Toutes les instructions sont exécutées dans l'ordre
- Cela constitue un **processus séquentiel**.



Exécution séquentielle d'un programme

2. Exécution parallèle

- Exemple : $i1 < (i2 \parallel i3) < i5 \parallel i4 < i6$
 - Certaines instructions s'exécutent en parallèle ($\parallel$)
 - Ce type d'exécution **n'est pas séquentiel**, c'est un **processus parallèle**.



Exécution parallèle du programme

2.2. Définition d'un processus séquentiel

- Un **processus séquentiel** est l'exécution strictement ordonnée d'instructions d'un programme (exécution a).
- Si certaines instructions peuvent s'exécuter en parallèle (exécution b), ce n'est pas un processus séquentiel. Dans ce cas, on peut décomposer l'exécution en **processus parallèles** :
 - P1 : $i1 < i3 < i5$

- $P2 : i2 < i4 < i6$

Différence entre programme et processus :

- **Programme** : entité statique, description d'un algorithme.
- **Processus** : entité dynamique, représentation abstraite de l'exécution séquentielle d'un programme.
- Un même programme peut générer plusieurs processus.

Exemples :

- **Informatique** : plusieurs utilisateurs compilant des programmes C → autant de processus compilateur C.
- **Non informatique** : préparation d'un gâteau comme processus séquentiel d'instructions.

Programme	Recette pour préparer le gâteau
Données	Ingrédients utilisés : Farine, beurre, sucre, amandes, ...
Processeur	Cuisinier
Processus	Actions de préparation du gâteau ou exécution des différentes étapes de la recette

Différence entre programme et processus

Complément :

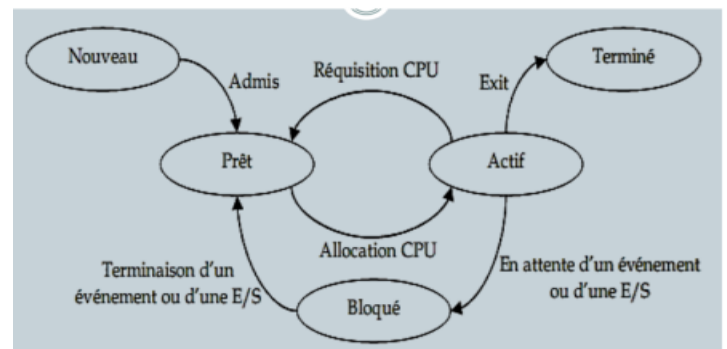
Un processus représente l'état dynamique du processeur et de la mémoire lors de l'exécution d'un programme, tandis que le programme lui est statique.

Fondamental :

Les systèmes d'exploitation cherchent à gérer équitablement les processus : ceux de même priorité partagent les ressources également, et les processus plus prioritaires sont servis en premier sans bloquer les autres moins prioritaires.

2.3. Principaux états d'un processus :

- **Prêt (Ready)** : Le processus a toutes les ressources sauf le processeur et attend son tour pour s'exécuter.
- **Actif (Running)** : Le processus dispose de toutes les ressources, y compris le processeur, et s'exécute.
- **Bloqué (Waiting)** : Le processus est suspendu, en attente d'une ressource ou d'un signal autre que le processeur.



États d'un processus

2.4. Transitions d'un processus :

Un processus change d'état en fonction de différents événements :

- Attente ou disponibilité d'une ressource ;
- Arrivée d'un événement attendu ;
- Intervention d'un processus plus prioritaire ;
- Exécution d'opérations comme les entrées/sorties ou la fin du processus.

(1) : Création du processus

(2) : Le processus est élu

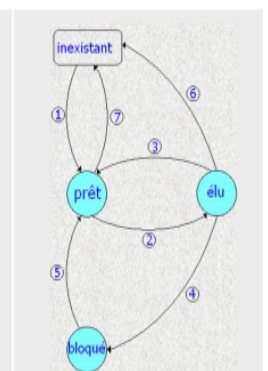
(3) : Fin du quantum de temps ou arrivée d'un processus plus prioritaire ;

(4) : Attente d'une ressource (physique ou logique) ;

(5) : Réveil du processus (libération d'une ressource ou arrivé d'un événement attendu) ;

(6) : Fin d'exécution du programme d'un processus (terminaison) ou erreur dans l'exécution du programme ;

(7) : Destruction du processus par un autre processus (le processus parent ou un processus système).



Transitions de processus

Structures de données pour les états des processus :

- **État prêt** : représenté par des files (globale, par priorité, ou par classe de processus : système, temps réel, temps partagé).
- **État bloqué** : une file par événement ou ressource attendue (ex. imprimante, fin d'E/S disque).
- **État actif** : une variable ou un tableau contenant les identifiants des processus en cours d'exécution.

2.5. Environnement d'un processus

2.5.1. Définition :

L'environnement d'un processus représente l'état complet du processus à un instant donné, incluant toutes les informations nécessaires pour son exécution et sa reprise après interruption, regroupant l'image mémoire et l'image processeur.

2.5.2. Image mémoire :

- **Segments de mémoire** :
 - **Segment de texte (Text Segment)** : Contient le code exécutable (lecture seule).
 - **Segment de données (Data Segment)** :
 - Données initialisées : variables globales/statique initialisées.
 - Données non initialisées (BSS) : variables globales/statique non initialisées.
 - **Segment de tas (Heap)** : Mémoire dynamique (malloc/new), croît vers les adresses supérieures.
 - **Segment de pile (Stack)** : Variables locales, paramètres, adresses de retour, croît vers les adresses inférieures.
- **Autres éléments en mémoire** :
 - Variables d'environnement et arguments passés au programme.
 - Bibliothèques partagées, chargées et utilisées par plusieurs processus.

2.5.3. Image processeur (Processor Image)

- **Définition** : Représente l'état du processeur pendant l'exécution d'un processus, incluant toutes les informations nécessaires pour reprendre l'exécution après interruption.
- **Éléments principaux** :
 - a) **Registres du processeur** :
 - **Compteur de programme (PC)** : Adresse de la prochaine instruction à exécuter.
 - **Registre d'état (Status Register)** : Indicateurs sur l'état du processeur (flags, mode noyau/utilisateur).
 - **Registres généraux** : Pour calculs et stockage temporaire de données.
 - **Pointeur de pile (SP)** : Pointe vers le sommet de la pile du processus.
 - b) **Contexte d'exécution** :
 - Lors d'un **changement de contexte**, le système sauvegarde l'état du processeur dans le **PCB (Process Control Block)**.
 - Permet de restaurer le processus lorsqu'il reprend l'exécution.

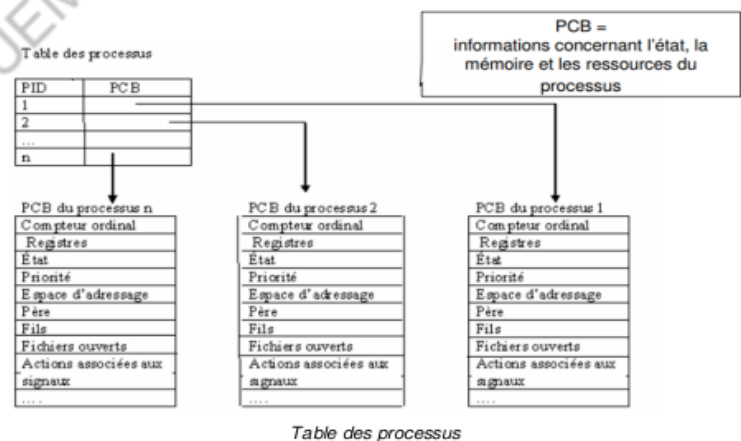
2.6. PCB – Process Control Block

- **Définition** : Structure définissant un processus dans le système, regroupant deux types d'informations :
 - **Contexte d'un processus** : Données permettant de reprendre l'exécution d'un processus interrompu, comprenant :
 1. Contexte du processeur (registres internes et externes)
 2. Espace mémoire utilisé par le programme et ses données
 3. Pile d'exécution
 - **Informations de gestion** : Pour identifier et gérer le processus, incluant :
 1. PID (identificateur unique) et nom symbolique
 2. Priorité et état du processus
 3. Relations avec père, fils, frères
 4. Fichiers ouverts et E/S en cours
 5. Comptabilité (temps CPU, E/S...)
 6. Variables d'environnement et autres informations
- **Commutation de contexte (Context Switching)** :
 - **Objectif** : Permettre à un système multitâches de passer d'un processus A à un processus B.
 - **Méthode** :
 1. **Sauvegarde** : L'état du processus A est enregistré dans son PCB (registres, compteur ordinal, etc.).
 2. **Chargement** : L'état du processus B est chargé depuis son PCB dans les registres du processeur.
 3. **Reprise** : Le processus B reprend l'exécution à partir de l'état restauré.

2.7. Tables des processus

Les PCB de tous les processus sont regroupés dans une **table des processus** (ou table des descripteurs de processus).
Chaque entrée de cette table contient généralement un **pointeur vers le PCB** du processus correspondant.

Remarque : La **table des processus** est conservée dans la mémoire du système d'exploitation et n'est **pas accessible aux processus** eux-mêmes.



2.8. Opérations sur les processus

2.8.1. Opérations sur les processus

Un processus est un objet identifié par un **identificateur unique** sur lequel le système d'exploitation peut effectuer différentes opérations :

- Création, destruction

- Blocage, réveil
- Activation (dispatcher), désactivation (fin de quantum)
- Suspension

Ces opérations constituent le **noyau des fonctions du système d'exploitation**.

2.8.2. Création d'un processus

La création d'un processus implique :

- Allocation d'un PCB (descripteur)
- Attribution d'un identificateur unique
- Initialisation du PCB : programme à exécuter, pile, mémoire, état, priorité et autres ressources

Complément : Qui peut créer des processus ?

- Un processus peut créer d'autres processus, qui peuvent eux-mêmes en créer.
- **MS-DOS** : Le processus créateur est suspendu jusqu'à la fin du processus créé (exécution séquentielle).
- **Windows** : Les processus créateurs et créés s'exécutent en concurrence, sans relation hiérarchique explicite (exécution asynchrone).

2.8.3. Destruction de processus

Méthode de destruction :

- Réalisée par le **processus père** ou par un **processus système** (UNIX/Linux)
- Libération des **ressources** occupées par le processus
- Destruction éventuelle de la **descendance** (selon le système ; sous UNIX, les fils ne sont pas détruits automatiquement)
- Libération du **descripteur PCB**

Remarque :

- Le PCB peut être **réutilisé** après destruction
- L'**identificateur du processus** ne peut pas être réutilisé.

3. Processus UNIX/LINUX

3.1. Arguments d'un programme

3.1.1. Arguments d'un programme

- Un programme exécutable peut recevoir des **arguments en ligne de commande** transmis par le shell lors du lancement.

La fonction `main()` en C peut être définie comme :

```
int main(int argc, char *argv[], char *arge[])
```

- **argc** : nombre de paramètres (y compris le nom du programme)
- **argv** : tableau des arguments passés en ligne de commande
- **arge** : tableau des variables d'environnement (**variable=valeur**)

- Les noms `argc`, `argv` et `arge` sont **conventionnels**.

Exemple

Voici un programme qui illustre l'utilisation des arguments `argc` et `argv`.

```
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char *argv[])
{
    int k;
    printf("Affichage des arguments :\n");

    for (k = 0; k < argc; k++)
    {
        printf("%d : %s\n", k, argv[k]);
    }
}
```

Voici un exemple de trace :

```
$ ./env1 a b c
```

Affichage des arguments:

```
0: ./env1
```

```
1: a
```

```
2: b
```

```
3: c
```

3.2. Communication avec l'environnement

3.2.1. Variables d'environnement

- **Définition** : Valeurs dynamiques en mémoire utilisées par plusieurs processus simultanément.
- **Syntaxe** : `NOM = VALEUR`
- **Accès en C** :

Via la variable globale externe `environ` :

```
extern char **environ;
```

1. Via la variable globale externe `environ` :
 - Contient des chaînes de caractères terminées par `NULL`
 - Le tableau lui-même se termine par un pointeur nul
2. Via le paramètre `arge[]` de la fonction `main()`

3.2.2. Principales variables d'environnement sous Linux

- `HOME` : répertoire personnel de l'utilisateur
- `PATH` : chemins des exécutables
- `PWD` : répertoire de travail courant

- **USER / LOGNAME** : nom de l'utilisateur
- **TERM** : type de terminal utilisé
- **SHELL** : shell de connexion utilisé

Exemple

Voici un programme qui illustre l'utilisation de l'argument arge :

```
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char *argv[], char *arge[])
{
    int k;
    printf("\nAffichage des variables d'environnement :\n");

    for (k = 0; arge[k] != NULL; k++)
    {
        printf("%d : %s\n", k, arge[k]);
    }
}
```

La trace de ce programme est la suivante :

Affichage des variables d'environnement:

0: PWD=/home/invite/TP_system/tp2

1: ...

36: SHELL=/bin/bash

41: HOME=/home/invite

42: TERM=xterm

43: PATH=/bin:/usr/bin:/usr/local/bin:/home/invite/bin

3.2.3. Manipulation des variables d'environnement en C

- Le comportement d'un programme peut dépendre du terminal, de l'utilisateur, du répertoire de travail ou des variables d'environnement.
- En C, on manipule ces variables avec des fonctions spécifiques.

1/Fonction de recherche : **getenv**

- Permet d'obtenir la valeur d'une variable d'environnement à partir de son nom.
- Déclaration :

```
#include <stdlib.h>
char* getenv(const char* name);
```

- **Paramètre** : **name** → nom de la variable recherchée
- **Retour** : pointeur vers la valeur de la variable si elle existe, sinon **NULL**

Exemple : Voici un programme qui illustre l'utilisation de l'appel système `getenv()`

```
#include <stdio.h>
#include <stdlib.h> // pour utiliser getenv()
```

```

int main(void)
{
    char *valeur;

    valeur = getenv("PATH"); // Récupère la valeur de la variable d'environnement PATH

    if (valeur != NULL)
        printf("Le PATH vaut : %s\n", valeur); // Affiche sa valeur
    return 0;
}

```

2/Fonction de Création : **putenv**

- Crée une variable d'environnement ou modifie sa valeur si elle existe.
- Syntaxe :

```

#include <stdlib.h>
int putenv(const char *string); // string = "NOM=VALEUR"

```

- Retour : 0 si succès, -1 si erreur.

3/Fonction de Modification : **setenv**

- Modifie ou crée une variable d'environnement selon le paramètre **rewrite**.
- Syntaxe :

```

#include <stdlib.h>
int setenv(const char *name, const char *value, int rewrite);

```

- **rewrite = 0** → crée seulement si la variable n'existe pas
- **rewrite != 0** → modifie la variable existante
- Retour : 0 si succès, -1 si erreur.

4/Fonction de Suppression : **unsetenv**

- Supprime une variable d'environnement si elle existe.
- Syntaxe :

```

#include <stdlib.h>
int unsetenv(const char *name);

```

- Retour : 0 si succès, -1 si erreur.

3.3. Appels système POSIX pour les processus sous Linux

- **Création** : lancer un nouveau processus.
- **Terminaison** : mettre fin à un processus existant.
- **Lancement** : exécuter un programme dans un processus.

3.3.1. La création de processus

a) Création de processus

- Utilise l'appel système `fork()` : `pid_t fork(void)`
- Processus 0 (Swapper) créé au démarrage du système.
- Processus 0 crée le processus init (PID=1), ancêtre de tous les autres.
- Tous les autres processus sont créés par `fork()` :
 - Le processus appelant est le **père**.
 - Le nouveau processus est le **fil**.
 - Chaque processus a un PID unique et un seul père, mais peut avoir plusieurs fils.
- Méthode `fork()` :
 - Alloue un PCB pour le fils.
 - Copie le PCB du père (sauf PID, PPID...).
 - Associe un PID au fils.
 - Segment de code attribué au fils ; segments données/pile utilisés en **copy-on-write**.
 - État du processus : **exécution**.

b) Terminaison de processus

- Utilise l'appel système `exit(int status)` : termine un processus volontairement.
- `status` : code de retour (0–255), communiqué au père.
- Convention : 0 = terminaison normale.

Exemple:voici un programme qui illustre l'utilisation de l'appel système exit()

```
#include <stdio.h>
#include <stdlib.h> // pour utiliser exit()

void quit(void)
{
    printf("Nous sommes dans la fonction quitterProgramme\n");
    exit(); // termine immédiatement le programme
}

int main(void)
{
    quit(); // appel de la fonction quit()
    printf("Nous sommes dans le main.\n");
    return 0;
}
```

Exemple

Voici un programme qui illustre l'utilisation de l'appel système exit() :

```
#include <stdio.h>
#include <stdlib.h> // pour utiliser exit()

void quit(void)
{
```

```

printf("Nous sommes dans la fonction quitter Programme\n");
exit(); // termine immédiatement le programme
}

```

```

int main(void)
{
    quit(); // appel de la fonction quit()
    printf("Nous sommes dans le main.\n");
    return 0;
}

```

La trace de ce programme est la suivante :

Nous sommes dans la fonction quitter Programme.

L'exécution montre que l'instruction `printf("Nous sommes dans le main.\n");` n'est pas exécutée.

Le processus s'est arrêté à la lecture de `exit()`.

c) Attente de terminaison d'un processus fils

- Si un **processus fils** se termine avant son père, il devient un **zombie** : terminé mais présent dans la table des processus.
- Le père doit récupérer le **code de retour** du fils pour libérer son entrée.
- Pour cela, le père utilise les primitives `wait()` ou `waitpid()` (bibliothèque `sys/wait.h`).

La primitive `wait()`

- Permet au **processus père** d'attendre la fin d'un processus fils.
- Syntaxe : `pid_t wait(int *status);`
- Retourne :
 - Le **PID** du fils terminé,
 - Son **état de terminaison** dans `status`.
- Retourne **-1** si tous les fils sont déjà terminés.

La primitive `waitpid()`

- Permet d'attendre la fin d'un **processus enfant spécifique** et de récupérer son statut.
- Syntaxe : `pid_t waitpid(pid_t pid, int *status, int options);`

Paramètres :

- `pid` : quel processus attendre
 - **0** → tous les enfants du même groupe de processus que le parent
 - **-1** → n'importe quel enfant terminé (comme `wait()`)
 - **>0** → attendre un enfant avec PID spécifique
- `status` : pointeur pour stocker le statut de fin
- `options` :
 - **0** → attente bloquante par défaut
 - **WNOHANG** → ne bloque pas si aucun enfant terminé
 - **WUNTRACED** → retourne aussi les processus stoppés
 - **WCONTINUED** → retourne aussi les processus repris

Valeur de retour :

- PID du processus terminé si succès
- 0 si **WNOHANG** et aucun enfant terminé
- -1 en cas d'erreur (aucun enfant)

Complément : Lecture du mot **status**

Pour décoder l'état de terminaison d'un processus enfant (**status**) après un **wait()** ou **waitpid()**, on utilise des macros :

- **WIFEXITED(status)** → vrai si l'enfant s'est terminé normalement
- **WEXITSTATUS(status)** → valeur retournée par l'enfant
- **WIFSIGNALED(status)** → vrai si l'enfant a été terminé par un signal
- **WTERMSIG(status)** → signal reçu par l'enfant
- **WIFSTOPPED(status)** → vrai si l'enfant est bloqué (nécessite **WUNTRACED**)
- **WSTOPSIG(status)** → signal bloquant reçu par l'enfant (nécessite **WUNTRACED**)

Complément : Interprétation de **status**

- Si le fils est stoppé :
 - 8 bits de poids fort → numéro du signal ayant arrêté le processus
 - 8 bits de poids faible → valeur octale 177
- Si le fils s'est terminé avec **exit()** :
 - 8 bits de poids faible → 0
 - 8 bits de poids fort → contient la valeur passée par le fils à **exit()**

Remarque : Comportement de **wait** et **waitpid**

- **wait(status)** et **waitpid(-1, status, 0)** → bloquent le processus parent jusqu'à ce qu'un de ses fils se termine.
- **waitpid(pid, status, 0)** avec **pid > 0** → bloque jusqu'à la terminaison du fils spécifique identifié par **pid**.
- **waitpid(pid, status, WNOHANG)** → ne bloque pas, retourne 0 si le fils n'est pas encore terminé.

Attention :

Si un processus parent meurt sans récupérer l'état de ses enfants, ces derniers deviennent des **orphelins**.

- Le noyau les confie au processus **init (PID 1)**, qui devient leur "parent adoptif".
- Le processus **init** appelle régulièrement **wait()** pour récupérer ces enfants et libérer les ressources, évitant l'accumulation de **zombies** et assurant la stabilité du système.

d) Mise en pause d'un processus : **sleep()**

- La fonction **sleep(nbSec)** suspend l'exécution du processus pendant le nombre de secondes spécifié.
- Valeurs de retour :

- Nombre de secondes restantes si le processus reçoit un signal.
- 0 si le délai s'écoule normalement.

e) Quelques exemples

Exemple 1 :

Voici un programme qui illustre l'utilisation des appels système fork(), getpid() , getppid(), et wait();

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h> // pour fork(), getpid(), getppid()
#include <sys/types.h> // pour pid_t
#include <sys/wait.h> // pour wait()

int main()
{
    int pid = fork(); // Création d'un processus fils

    if (pid < 0) { // Erreur lors de la création
        perror("Problème de création par fork()");
        exit(EXIT_FAILURE);
    }

    if (pid > 0) { // Processus père
        wait(NULL); // Attend que le fils se termine
        printf("Je suis le processus père : pid=%d, mon pid=%d, ppid=%d\n", pid, getpid(), getppid());
    }

    if (pid == 0) { // Processus fils
        printf("Je suis le processus fils : pid=%d, mon pid=%d, ppid=%d\n", pid, getpid(), getppid());
    }

    system("ps -l"); // Affiche la liste des processus

    return 0;
}
```

La trace de ce programme est la suivante :

```
Je suis le processus fils : pid=0, mon pid=706, ppid=705
F S  UID  PID  PPID  C  PRI  NI  ADDR  SZ  WCHAN  TTY          TIME CMD
0 S  2001   328     8  0  98  18 - 2000  do_wai pts/0      00:00:00 bash
0 S  2001   705   328  0  98  18 - 591   do_wai pts/0      00:00:00 exemple1
1 S  2001   706   705  0  98  18 - 624   do_wai pts/0      00:00:00 exemple1
0 S  2001   707   706  0  98  18 - 654   do_wai pts/0      00:00:00 sh
0 R  2001   708   707  0  98  18 - 1893  -      pts/0      00:00:00 ps
Je suis le processus père : pid=706, mon pid=705, ppid=328
F S  UID  PID  PPID  C  PRI  NI  ADDR  SZ  WCHAN  TTY          TIME CMD
0 S  2001   328     8  0  98  18 - 2000  do_wai pts/0      00:00:00 bash
0 S  2001   705   328  0  98  18 - 624   do_wai pts/0      00:00:00 exemple1
0 S  2001   709   705  0  98  18 - 654   do_wai pts/0      00:00:00 sh
0 R  2001   710   709  0  98  18 - 1893  -      pts/0      00:00:00 ps
```

Exemple 2

Voici un programme qui illustre l'utilisation des appels système fork(), getpid() , et wait() et exit();

```
#include <stdio.h>
#include <unistd.h> // pour fork(), getpid()
#include <stdlib.h> // pour exit()
#include <sys/wait.h> // pour wait()

int main()
{
    int n, m;

    if (fork() == 0) // Processus fils
```



```

{
    printf("\nJe suis le fils, mon pid = %d\n", getpid());
    exit(0); // Fin du processus fils
}
else // Processus père
{
    printf("Je suis le père, mon pid = %d\n", getpid());
    m = wait(&n); // Le père attend que le fils se termine

    printf("Fin de processus de pid = %d avec valeur de paramètre de wait = %d\n", m); // Affiche le
    PID du fils et sa valeur de sortie
}
}
}

```

La trace de ce programme est la suivante :

je suis le père, mon pid = 3912

Je suis le fils mon pid = 3913

Fin de processus de pid = 3913 avec valeur de retour de wait = 0

3.3.2. Lancement d'un programme : Les primitives de recouvrement exec

a) Lancement d'un programme avec **exec**

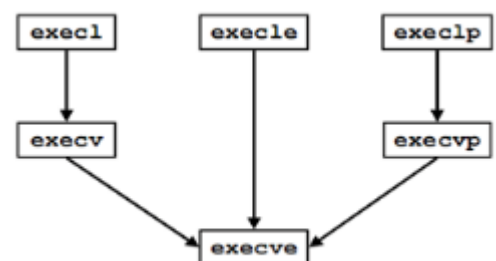
- Les primitives **exec** remplacent l'image mémoire d'un processus par un nouveau programme tout en conservant le PID.
- Méthode courante :
 - **fork()** pour créer un processus fils.
 - Dans le fils, appel à **exec** pour lancer un nouveau programme.
- Étapes du recouvrement :
 - Vérifier le fichier exécutable.
 - Remplacer le code existant par le nouveau.
 - Passer les paramètres au programme.
 - Générer un nouveau segment de données et vider la pile.
 - Mettre à jour le PCB (mémoire, compteur ordinal...)

Complément :

Les différentes primitives **exec** ont des variantes selon leur syntaxe et utilisation.

Complément sur les variantes de **exec**

- **p** : Cherche le programme dans le **PATH** (**execvp**, **execclp**). Sans **p**, il faut fournir le chemin complet.
- **v** : Reçoit les arguments sous forme de tableau de chaînes (**execv**, **execvp**, **execve**).
- **l** : Reçoit les arguments en liste variable (**execl**, **execclp**, **execle**).
- **e** : Permet de passer un tableau de variables d'environnement (**execve**, **execle**), chaque chaîne de la forme "**VARIABLE=**valeur".



Les appels système (primitives) de la famille **exec**

Fondamental :

1. `execl`

`int execl(char *ref, char *arg0, ..., char *argn, NULL);`

- **Usage** : Permet de passer un nombre **fixe** d'arguments au nouveau programme.
- **Paramètres** :
 - `ref` : chemin absolu du programme à exécuter.
 - `arg0, arg1, ..., argn` : arguments du programme (`arg0` = nom du programme).
 - Terminé par `NULL`.
- **Remarque** : Le processus courant est remplacé par le nouveau programme.

2. `execlp`

`int execlp(char *file, char *argv, ...);`

- **Usage** : Identique à `execl`, mais si `file` n'est pas un chemin absolu, recherche le programme dans les répertoires définis dans la variable `PATH`.
- **Paramètres** : Comme `execl`

3. `execv`

`int execv(char *ref, char *argv[]);`

- **Usage** : Permet de passer un nombre **variable** d'arguments via un tableau `argv[]`.
- **Paramètres** :
 - `ref` : chemin du programme.
 - `argv[]` : tableau de chaînes de caractères contenant les arguments, terminé par `NULL`.

4. `execve`

`int execve(char *ref, char *argv[], char *envp[]);`

- **Usage** : Remplace le processus courant par un nouveau programme, avec **arguments** et **environnement** spécifiés.
- **Paramètres** :
 - `ref` : chemin du programme.
 - `argv[]` : arguments du programme, terminé par `NULL`.
 - `envp[]` : tableau de chaînes de caractères pour l'environnement, chaque chaîne = "VARIABLE=valeur", terminé par `NULL`.

5. `execle`

`int execle(char *ref, char *argv, char *envp[]);`

- **Usage** : Comme `execl`, mais permet de spécifier un environnement `envp[]`.
- **Paramètres** :
 - `ref` : chemin du programme.
 - `argv[]` : arguments, terminé par `NULL`.
 - `envp[]` : tableau pour l'environnement, terminé par `NULL`.

6. `execvp`

`int execvp(char *file, char *argv[]);`

- **Usage** : Comme `execv`, mais si `file` n'est pas un chemin complet, recherche dans les répertoires de `PATH`.
- **Paramètres** :
 - `file` : nom du programme.
 - `argv[]` : arguments, terminé par `NULL`.

Résumé général :

- `l` → arguments passés sous forme de liste (`arg0, arg1, ...`).
- `v` → arguments passés sous forme de **tableau** (`argv[]`).
- `p` → recherche le programme dans `PATH` si nom pas complet.
- `e` → permet de passer un **tableau d'environnement** (`envp[]`).

Attention

Après un appel à une fonction de la famille `exec`, **le processus courant est remplacé par le nouveau programme**, donc **le code qui suit l'appel ne s'exécute pas**.

Il ne s'exécute **que si l'exécution du nouveau programme échoue** (par exemple si le chemin est incorrect).

Exemple

Voici un programme qui illustre l'utilisation des appels système `fork()` et `execl()` :

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h> // pour fork(), getpid(), execl()
#include <sys/wait.h> // pour wait()

int main()
{
    int pid, status;

    pid = fork(); // Création du processus fils

    if (pid > 0) // ----- Processus PÈRE -----
    {
        wait(&status); // Attend la fin du fils
        printf("Je suis le processus père : pid fils = %d, mon pid = %d, pid parent = %d\n",
            pid, getpid(), getppid());
    }
    if (pid == 0) // ----- Processus FILS -----
    {
        printf("Je suis le processus fils : %d-%d-%d\n", pid, getpid(), getppid());
    }
}
```

```

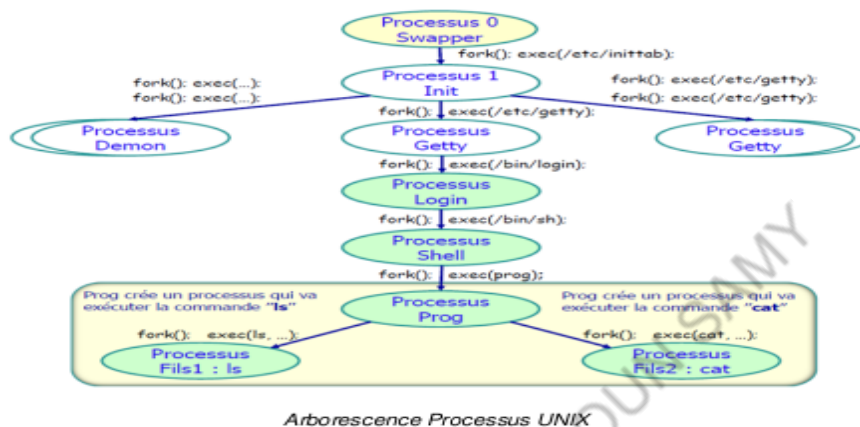
// Remplace le code du fils par la commande "ls -l"
execl("/bin/ls", "ls", "-l", NULL);

}

if (pid < 0) // ----- ERREUR de fork -----
{
    perror("Problème de création par fork()");
}
}
}

```

Arborescence Processus UNIX



II Threads

Les threads (fils d'exécution) permettent à un programme d'exécuter plusieurs tâches en parallèle, améliorant ainsi performances et réactivité. Ils constituent un concept clé en programmation concurrente et impliquent des notions de gestion et de problématiques spécifiques.

1. Définition

Un **thread** est une séquence d'instructions pouvant s'exécuter indépendamment. Dans un programme multi-thread, plusieurs threads s'exécutent **simultanément** et **partagent les ressources** du processus (mémoire, fichiers, etc.).

Différence avec un processus :

- **Processus** : instance d'un programme avec sa propre mémoire et ressources.
- **Thread** : unité d'exécution à l'intérieur d'un processus, partageant la mémoire et les ressources avec les autres threads du même processus.

2. Opérations sur les threads

Création de Threads

La fonction `pthread_create` permet de **créer un nouveau thread** qui exécute une fonction spécifique avec un argument donné.

- Syntaxe : `int pthread_create(pthread_t *thread, pthread_attr_t *attr, void *nomFonct, void *arg);`
- Retour : 0 si succès, sinon une valeur d'erreur.
- `*thread` : stocke l'identifiant du thread (TID).
- `*attr` : définit les attributs du thread (pile, priorité, planification); `NULL` par défaut.

Attente de terminaison d'un thread

La fonction `pthread_join` permet à un thread d'attendre la fin d'un autre thread, similaire à `waitpid` pour les processus.

- Syntaxe : `void pthread_join(pthread_t tid, void **status);`
- `tid` : identifiant du thread à attendre.
- `status` : récupère la valeur de retour et l'état de terminaison du thread.

Identification du TID d'un thread

La fonction `pthread_self()` retourne l'identifiant (TID) du thread courant.

- Syntaxe : `pthread_t pthread_self(void);`

Terminaison de threads

La fonction `pthread_exit()` termine l'exécution d'un thread et permet de retourner un état de terminaison.

- Syntaxe : `void pthread_exit(void *valeur_de_retour);`
- La valeur de retour est récupérée par `pthread_join()`.

Exemple 1

Voici un programme qui illustre l'utilisation des primitives liées à la gestion de threads :

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <sys/types.h>
#include <unistd.h>

void *helloworld(void *arg)
{
    printf("Hello world! pid = %d, pthread_self = %p\n", getpid(), (void *)pthread_self());
    return NULL;
}

int main()
{
    pthread_t thread1, thread2;

    // Création des deux threads
    pthread_create(&thread1, NULL, helloworld, NULL);
    pthread_create(&thread2, NULL, helloworld, NULL);
```

```

// Attente de la fin des threads
pthread_join(thread1, NULL);
pthread_join(thread2, NULL);

return 0;
}

```

La trace de ce programme après compilation est la suivante :

```

ubuntu@ubuntu-VirtualBox:~$ cc exo.c -lpthread
ubuntu@ubuntu-VirtualBox:~$ ./a.out

```

```

Hello world! pid=14714 pthread_self=0xb6579b40
Hello world! pid=14714 pthread_self=0xb6d7ab40

```

Exemple 2

```

#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h> // Pour la fonction usleep()

// Fonction exécutée par chaque thread
void* fonction_thread(void* arg) {
    int id = *(int*)arg; // Récupérer l'identifiant du thread

    printf(" [Thread %d] Démarrage...\n", id);

    // Simulation de travail avec des affichages progressifs
    for (int i = 1; i <= 3; i++) {
        printf(" [Thread %d] Étape %d...\n", id, i);
        usleep(500000); // Pause de 500 millisecondes
    }

    printf(" [Thread %d] Terminé !\n", id);
    pthread_exit(NULL); // Fin propre du thread
}

int main() {
    int nb_threads = 3;           // Nombre de threads
    pthread_t threads[nb_threads]; // Tableau des threads
    int ids[nb_threads];          // Identifiants des threads

    // Création des threads
    for (int i = 0; i < nb_threads; i++) {
        ids[i] = i + 1;
        printf(" Création du thread %d...\n", ids[i]);
        if (pthread_create(&threads[i], NULL, fonction_thread, &ids[i]) != 0) {
            perror(" Erreur création thread");
            return EXIT_FAILURE;
        }
    }
}

```

```

    }

    // Attente de la fin des threads
    for (int i = 0; i < nb_threads; i++) {
        pthread_join(threads[i], NULL);
    }
    printf(" Tous les threads ont terminé leur travail !\n");
    return EXIT_SUCCESS;
}

```

voici une trace de l'exécution de ce programme :

```

✖Création du thread 1...
✖Création du thread 2...
◆[Thread 1] Démarrage...
⌚[Thread 1] Étape 1..
✖Création du thread 3...
◆[Thread 2] Démarrage...
⌚[Thread 2] Étape 1..
◆[Thread 3] Démarrage...
⌚[Thread 3] Étape 1..
⌚[Thread 1] Étape 2..
⌚[Thread 2] Étape 2..
⌚[Thread 3] Étape 2..
⌚[Thread 1] Étape 3..
⌚[Thread 2] Étape 3..
⌚[Thread 3] Étape 3..
✅[Thread 1] Terminé !
✅[Thread 2] Terminé !
✅[Thread 3] Terminé !

```

MAGUEMOUN SAMY

Chapitre 4: Les tubes

Le système d'exploitation gère la **communication entre processus** dans un environnement multitâche où chaque processus a sa propre mémoire. Pour échanger des données malgré cette isolation, le SE propose plusieurs mécanismes d'IPC :

- **Tubes (pipes)**
- **Files de messages**
- **Segments de mémoire partagée**
- **Signaux**

I/Définition des tubes de communications (Pipes) :

Les **pipes** sont un mécanisme de communication inter-processus (IPC) qui permet à deux processus d'échanger des données. Très utilisés sous Unix/Linux et Windows, ils servent à faire circuler des informations entre processus pour qu'ils puissent collaborer.

1. Descripteurs de fichiers

Un **descripteur de fichier** est un identifiant permettant à un processus d'accéder à un fichier ou à une ressource d'E/S. Il correspond à une entrée dans la table des descripteurs (dans le PCB) et indique les informations nécessaires pour manipuler ce fichier (mode d'accès, position, etc.).

Méthode

Lorsque le processus ouvre un fichier :

- le noyau crée un descripteur de fichier et l'ajoute dans la table des descripteurs du processus ;
- ce descripteur est associé à un buffer en mémoire ;
- le processus utilise ce descripteur avec les appels système **read()**, **write()**, **close()**.

Remarque

Toutes les opérations d'entrée/sortie utilisant ce descripteur passent par le buffer, qui stocke temporairement les données lues ou écrites.

2. Flots d'entrée /sortie

Définition

Les flots d'entrée/sortie sont des canaux qui permettent à un programme d'échanger des données avec l'extérieur (fichiers, clavier, écran, etc.). Chaque flot est une séquence de données permettant la lecture ou l'écriture.

Catégories

- **Flux d'entrée** : utilisés pour recevoir des données (clavier, lecture de fichier).
- **Flux de sortie** : utilisés pour envoyer des données (écran, écriture dans un fichier).

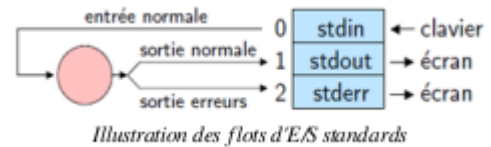
Remarque

Chaque flot repose sur un descripteur de fichier. À la création d'un processus, trois descripteurs standards sont automatiquement fournis.

Descripteur	Nom	Usage	Flux
0	stdin (Standard Input)	Entrée standard (clavier, fichier, etc.)	Lecture
1	stdout (Standard Output)	Sortie standard (écran, fichier, etc.)	Écriture
2	stderr (Standard Error)	Sortie des erreurs	Écriture

Flux d'E/S standards

Ces descripteurs sont automatiquement créés et associés à des buffers en mémoire pour gérer l'entrée et la sortie du processus.



3. Les redirections

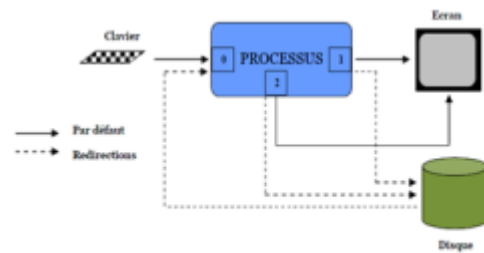
3.1. Les redirections

Les redirections permettent de changer la source ou la destination des flots d'E/S d'un processus.

On peut fermer ou rediriger les flux vers de vrais fichiers.

Syntaxe

- `<` : redirection de l'entrée.
- `>`, `>>`, `2>` : redirection de la sortie.



3.2. Redirection des entrées-sorties standards

Lorsqu'un processus est créé, ses entrées/sorties standards peuvent être redirigées vers de vrais fichiers (ou des tubes).

La sortie standard peut être envoyée dans un fichier soit en l'écrasant, soit en ajoutant à la fin s'il existe déjà.

Règles essentielles

- `> fichier` : redirige la sortie standard vers *fichier*, qui est créé ou écrasé.
- Permet d'envoyer directement le résultat d'une commande dans un fichier.

Exemple :

L'exécution de la commande redirige la sortie standard vers un fichier nommé **listing**.

En plus des fichiers déjà présents (« groupe » et « section »), le système crée donc automatiquement le fichier **listing** dans le répertoire *info*. « `ls > listing` »

```
ubunu@ubunu-VirtualBox:~/info $ ls
```

```
groupe
```

```
section
```

```
ubunu@ubunu-VirtualBox:~/info $ ls > listing
```

```
ubunu@ubunu-VirtualBox:~/info $ cat listing
```

```
groupe
```

```
listing
```

```
section
```

Si on utilise le double caractère ">>", l'information ou la redirection (sortie standard) sera ajoutée à la fin du

fichier sans écraser son contenu.

Exemple :

```
ubunu@ubunu-VirtualBox:~/info$ cat listing
```

groupe

listing

Section

```
ubunu@ubunu-VirtualBox:~/info$ echo "Bonjour" >> listing
```

```
ubunu@ubunu-VirtualBox:~/info$ cat listing
```

groupe

listing

section

Bonjour

La redirection des entrées

La redirection de l'entrée standard permet à une commande de lire ses données depuis un fichier au lieu du clavier.

Elle utilise le symbole "<", qui doit être suivi d'un fichier déjà existant.

Dans ce cas, la commande prend son entrée directement depuis ce fichier.

exemple :

```
ubunu@ubunu-VirtualBox:~/info$ cat listing
```

Groupe

listing

section

Bonjour

```
ls: cannot access fiche: No such file or directory
```

```
ubunu@ubunu-VirtualBox:~/info$ sort < listing
```

Bonjour

Groupe

listing

section

ls: cannot access fiche: No such file or directory section

La redirection des sorties d'erreur

Le caractère "2>" suivie d'un nom de fichier indique la redirection de la sortie d'erreur vers ce fichier.

Exemple :

```
ubunu@ubunu-VirtualBox:~/info$ ls fiche
```

```
ls: cannot access fiche: No such file or directory
```

```
ubunu@ubunu-VirtualBox:~/info$ ls fiche 2>erreur
```

```
ubunu@ubunu-VirtualBox:~/info$ cat erreur
```

```
ls: cannot access fiche: No such file or directory
```

Si on ne veut pas écraser le fichier, mais ajouter à sa fin, on utilise (2>>fichier).

```
ubunu@ubunu-VirtualBox:~/info$ ls fiche 2>>listing
```

```
ubunu@ubunu-VirtualBox:~/info$ cat listing
```

groupe

listing

section

Bonjour

```
ls: cannot access fiche: No such file or directory
```

4. Les tubes de communications (Pipes)

4.1. Définition des tubes de communications (Pipes)

Les **tubes (pipes)** sont des canaux de communication **unidirectionnels** qui permettent à plusieurs processus d'une même machine d'échanger des données sous forme d'un **flux FIFO** (First In, First Out).

Points essentiels :

- Les pipes sont un mécanisme IPC simple et très utilisé.
- Ils fonctionnent en redirigeant la **sortie standard** d'un processus vers l'**entrée standard** d'un autre.
- Ils servent fréquemment à faire communiquer un **processus parent** et un **processus fils**.

Exemple :

Le shell utilise le symbole `|` pour créer un tube entre deux commandes.

La commande suivante crée une chaîne de trois processus reliés par des pipes :

```
cat f1 f2 | grep bon | wc -l
```

Fonctionnement :

1. `cat f1 f2` envoie le contenu des fichiers dans un pipe.
2. `grep bon` lit depuis ce pipe, filtre les lignes contenant "bon", et envoie le résultat dans un second pipe.
3. `wc -l` lit depuis ce second pipe et compte le nombre de lignes.

```
ubunu@ubunu-VirtualBox:~/info$ cat>f1
```

```
bon bonjour salut
```

```
ubunu@ubunu-VirtualBox:~/info$ cat>f2
```

```
rien bonBon
```

```
ubunu@ubunu-VirtualBox:~/info$ cat f1 f2
```

```
bon bonjour salut rien bonBon
```

```
ubunu@ubunu-VirtualBox:~/info$ cat f1 f2|grep bon
```

```
bon bonjour bonBon
```

```
ubunu@ubunu-VirtualBox:~/info$ cat f1 f2|grep bon|wc -l
```

```
3
```

4.2. Types de tubes

Il existe deux catégories de tubes :

- **Pipes anonymes** : utilisés pour communiquer entre processus ayant un ancêtre commun, généralement un parent et ses fils.
- **Pipes nommés (FIFO)** : possèdent un chemin dans le système de fichiers et permettent la communication entre processus indépendants via un fichier spécial.

Remarque

Tous les tubes sont gérés comme des fichiers par le système : ils possèdent un **i-node**, même lorsque le pipe est anonyme et n'a pas de nom dans le système de fichiers.

4.3. Les tubes anonymes (unnamed pipe)

Définition

- Les tubes anonymes (unnamed pipes) sont des canaux de communication unidirectionnels accessibles uniquement aux processus ayant un lien parent-enfant.
- La taille maximale d'un tube varie selon la version d'Unix, environ **4 Ko** (**PIPE_BUF**).

Méthode : Création d'un tube anonyme

- La primitive utilisée est :

```
#include <unistd.h>
```

```
int pipe(int desc[2]);
```

- Après création, **pipe()** fournit deux descripteurs :
 - **desc[0]** : lecture du tube
 - **desc[1]** : écriture dans le tube
- Seuls le processus créateur et ses descendants peuvent utiliser le tube.
- Valeur de retour :
 - **0** si succès
 - **-1** si échec (manque d'espace)

Exemple

```
#include <unistd.h>
```

```
int p[2];
```

```
pipe(p);
```

```
// p[0] : descripteur de lecture
```

```
// p[1] : descripteur d'écriture
```

ou de manière plus compacte :

```
#include <unistd.h>
```

```
int pipe(int p[2]);
```

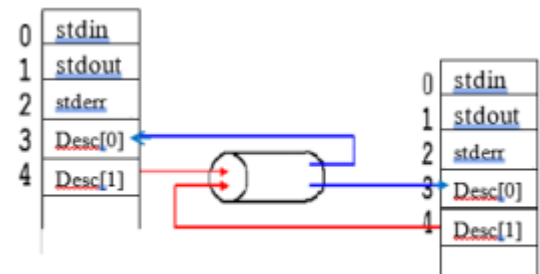
Complément : Fonctionnement des tubes anonymes

1. Le tube est créé par l'appel système **pipe()**.
2. Le processus créateur crée un processus fils via **fork()**.

3. Le descripteur du tube est dupliqué dans le processus fils, car la table des descripteurs fait partie du PCB du père.
4. Chaque tube a deux accès possibles :
 - Un pour le père, un pour le fils
 - Chaque processus choisit le sens de communication sur le tube commun

Communication

- La coordination entre père et fils est essentielle pour éviter le blocage (deadlock).
- Les processus échangent des données via les appels `read()` et `write()` sur le pipe commun.



Communication des processus père et fils avec les tubes

Méthode : Écriture dans un tube anonyme

- La primitive utilisée : `write()`
- **Syntaxe** : `int write(int desc[0], char *buf, int nb);`
- **Paramètres** :
 - `desc[0]` : descripteur du tube pour lecture
 - `buf` : tampon contenant les données à écrire
 - `nb` : nombre de caractères à écrire
- **Fonctionnement** :
 - Écrit `nb` caractères du tampon `buf` dans le tube
 - Retourne le nombre réel de caractères écrits
- **Exemple** :

```
int nb_lu; /* nombre de caractères lus */

nb_lu = write(p[0], buffer, TAILLE_READ);
```

Méthode : Fermeture d'un tube anonyme

- La primitive utilisée : `close()`
- **Syntaxe** : `int close(int fd);`
- **Fonctionnement** :
 - Ferme un descripteur de tube (`fd`) en lecture ou en écriture
 - Un tube est considéré fermé lorsque **tous ses descripteurs** (lecture et écriture) sont fermés
- **Remarque** :
 - La fermeture permet de libérer les ressources associées au tube et de signaler la fin des communications

Exemple d'utilisation d'un tube anonyme :

```
#include <stdio.h>

#include <stdio.h>

#include <unistd.h>
```

```

main(){
int desc_tube[2];

pipe(desc_tube); /* pipe(int tube[2]); */

char buf[20];

if (fork())

{ /* pere */

close (desc_tube[0]);

write (desc_tube[1], "bonjour", 8);}

else

{ /* fils */

close (desc_tube[1]);

read (desc_tube[0], buf, 8);

printf ("J'ai bien reçu votre message: %s \n", buf);}}

```

La trace de ce programme est la suivante :

\$ J'ai bien reçu votre message: bonjour

Exemple :

```

#include <stdio.h>

#include <unistd.h>

#include <stdlib.h>

int main() {

int fd[2]; // fd[0] pour la lecture, fd[1] pour l'écriture

pid_t pid;

char buffer[100];

if (pipe(fd) == -1) {

perror("pipe failed");

exit(EXIT_FAILURE);

```

```

}

pid = fork();

if (pid < 0) {

perror("fork failed");

exit(EXIT_FAILURE);

}

if (pid == 0) { // Processus enfant

close(fd[0]); // Ferme la lecture

write(fd[1], "Bonjour, parent!", 16);

close(fd[1]);

} else { // Processus parent

close(fd[1]); // Ferme l'écriture

read(fd[0], buffer, sizeof(buffer));

printf("Message reçu : %s\n", buffer);

close(fd[0]);}

return 0;}

```

La trace de e programme est :

Message reçu : Bonjour, parent!

Avantages et inconvénients des tubes anonymes

Avantages :

- Simple à utiliser
- Utilisation efficace de la mémoire grâce aux tampons internes
- Idéal pour la communication entre un processus parent et son fils
- Gestion automatique par le système (descripteurs fermés à la fin du processus)

Inconvénients :

- Unidirectionnels (nécessité de deux pipes pour communication bidirectionnelle)
- Limités aux processus liés par parenté
- Données non persistantes (disparaissent après lecture)

4.4. Les tubes nommés (Named Pipe)

Définition :

Un tube nommé (FIFO) est un mécanisme IPC qui permet à des processus **indépendants** d'échanger des données via un fichier spécial. Les données sont lues dans le même ordre qu'elles sont écrites (principe FIFO).

Caractéristiques :

- **Persistence** : Le tube reste dans le système de fichiers même si aucun processus ne l'utilise ; il doit être supprimé manuellement.
- **Communication entre processus indépendants** : Contrairement aux pipes anonymes, pas besoin de lien parent-enfant.
- **Unidirectionnel ou bidirectionnel** : Naturellement unidirectionnel ; deux FIFOs permettent une communication bidirectionnelle.
- **Synchronisation automatique** : Lecture/écriture bloquantes par défaut.
- **Gestion via le système de fichiers** : Manipulable comme un fichier classique (ls, rm, etc.).

Création :

- En ligne de commande
- Via la primitive **mkfifo()** en C.

Création d'un tube nommé en ligne de commande

Méthode :

On peut créer un tube nommé en utilisant les commandes **mkfifo** ou **mknod** dans le shell.

Exemple :

```
$ mknod groupe1 p    # Création d'un FIFO avec mknod
```

```
$ mkfifo groupe2    # Création d'un FIFO avec mkfifo
```

```
$ ls -l
```

```
prw-r--r-- 1 ..... groupe1
```

```
prw-r--r-- 1 ..... groupe2
```

Les fichiers créés apparaissent avec le type spécial **prw** indiquant qu'il s'agit de tubes nommés.

Création d'un tube nommé dans un programme

Méthode :

Un tube nommé peut être créé en C avec la primitive **mkfifo()**, qui renvoie **0** en cas de succès et **-1** en cas d'erreur.

Prototype :

```
#include <sys/types.h>
```

```
#include <sys/stat.h>
```

```
int mkfifo(char *nom, mode_t mode);
```

Paramètres :

- **nom** : nom du tube à créer.
- **mode** : permissions d'accès au tube, définies en octal (ex. 0644), représentant les droits pour le propriétaire, le groupe et les autres utilisateurs.

Les droits peuvent être exprimés en binaire pour chaque catégorie d'utilisateur afin de préciser les permissions de lecture, écriture et exécution.

Exemple et utilisation d'un tube nommé

Création avec permissions numériques :

```
mkfifo("test_tube.fifo", 0644);
```

- Crée le tube nommé **test_tube.fifo**.
- Permissions :
 - Propriétaire : lecture + écriture (6 → 110 en binaire)
 - Groupe : lecture (4 → 100)
 - Autres utilisateurs : lecture (4 → 100)

octal	binaire	droits
0	000	---
1	001	--x
2	010	-w-
3	011	-wx
4	100	r--
5	101	r-x
6	110	rw-
7	111	rwX

Droits d'accès en binaire et en octal

Création avec constantes symboliques :

```
mkfifo("test_tube.fifo", S_IRUSR | S_IWUSR | S_IRGRP | S_IROTH);
```

- Propriétaire : lecture + écriture
- Groupe : lecture
- Autres : lecture

Constantes pour permissions :

- **Propriétaire** : **S_IRUSR**, **S_IWUSR**, **S_IXUSR**, **S_IRWXU**
- **Groupe** : **S_IRGRP**, **S_IWGRP**, **S_IXGRP**, **S_IRWXG**
- **Autres** : **S_IROTH**, **S_IWOTH**, **S_IXOTH**, **S_IRWXO**

Utilisation :

- Ouvrir le tube avec **open()**
- Lire ou écrire avec **read()** et **write()**
- Fermer avec **close()**

Les tubes nommés se manipulent comme des fichiers classiques, mais permettent la communication entre processus indépendants.

Méthode : Ouverture d'un tube nommé

Prototype :

```
int open(const char *chemin_nom, mode_t mode_ouverture);
```

Paramètres :

- **chemin_nom** : nom complet du tube nommé (FIFO)
- **mode_ouverture** : mode d'accès, défini par les constantes `<sys/fcntl.h>` :
 - **O_RDONLY** : lecture seule (bloque si aucun écrivain)
 - **O_WRONLY** : écriture seule (bloque si aucun lecteur)
 - **O_RDWR** : lecture et écriture
 - **O_NONBLOCK** : mode non bloquant (ne suspend pas le processus si aucun lecteur/écrivain)

Retour :

- Succès : retourne un descripteur de fichier pour accéder au tube
- Échec : retourne **-1**

Cette primitive permet à un processus d'accéder à un tube nommé pour y lire ou écrire des données.

Méthode : Écriture dans un tube nommé

Prototype :

```
int write(int desc, char *buf, int nb);
```

Paramètres :

- **desc** : descripteur du FIFO obtenu avec `open()`
- **buf** : adresse du buffer contenant les données à écrire
- **nb** : nombre d'octets à écrire

Retour :

- Succès : retourne le nombre d'octets effectivement écrits
- Échec : retourne **-1**

Cette primitive permet d'envoyer des données dans un tube nommé pour qu'un autre processus puisse les lire.

Méthode : Lecture dans un tube nommé

Prototype :

```
int read(int desc, char *buf, int nb);
```

Paramètres :

- **desc** : descripteur du FIFO obtenu avec **open()**
- **buf** : buffer où stocker les données lues
- **nb** : nombre d'octets à lire

Retour :

- Nombre d'octets lus effectivement
- **0** si l'écrivain a fermé le FIFO
- **-1** en cas d'erreur

Méthode : Fermeture d'un tube nommé

Prototype : `int close(int fd);`

Paramètres :

- **fd** : descripteur de fichier à fermer

Retour :

- **0** en cas de succès
- **-1** en cas d'échec

Utilité de **close() :**

- Libère le descripteur de fichier
- Informe l'autre processus qu'il n'y aura plus de lecture/écriture
- Si l'écrivain ferme le FIFO, **read()** chez le lecteur retourne **0**

Exemple :

```
int fd = open("/tmp/myfifo", O_RDONLY);

if (fd != -1) {

    close(fd); // Ferme le FIFO après lecture

}
```

Méthode : Suppression d'un tube nommé avec **unlink()**

Prototype : `int unlink(const char *pathname);`

Paramètres :

- **pathname** : chemin du FIFO à supprimer

Retour :

- 0 en cas de succès
- -1 en cas d'échec

Remarques :

- Supprime le FIFO du système de fichiers
- Les processus qui l'ont déjà ouvert peuvent continuer à l'utiliser jusqu'à fermeture
- Après suppression, toute tentative de `open()` échoue

Avantages

- Permet la communication entre processus indépendants
- Disponible comme fichier spécial dans le système de fichiers
- Peut être bidirectionnel (souvent utilisé unidirectionnel)
- Persistant : reste accessible même après la fin des processus

Inconvénients

- Moins performant que les pipes anonymes (accès au système de fichiers)
- Risque d'encombrement si plusieurs processus accèdent simultanément
- Doit être créé et supprimé explicitement

Exemple d'utilisation d'un tube nommé :

```
/* Processus rédacteur sur le tube nommé */
```

```
#include<stdio.h>
```

```
#include<fcntl.h>
```

```
#include<sys/types.h>
```

```
#include<sys/stat.h>
```

```
#include<unistd.h>
```

```
#include<string.h>
```

```
int main (int argc, char *argv[])
```

```
{ int desc_tub;
```

```
const char *message = "message";
```

```
mkfifo("tube", 0777); /*création du tube nommé */
```

```
desc_tub = open ("tube", O_WRONLY ); /*ouverture du tube*/
```

```
write(desc_tub, message, strlen(message) + 1); /* écriture dans le tube */
```

```
printf("%d--fin d'écriture ",getpid());}
```

```
/* Processus lecteur sur le tube nommé */
```

```
#include<stdio.h>
```

```
#include<fcntl.h>
```

```
#include<sys/types.h>
```

```
#include<sys/stat.h>
```

```
#include<unistd.h>
```

```
int main ()
```

```
{ char buf [40];
```

```
int desc_tub;
```

```
desc_tub = open ("tube", O_RDONLY); /*ouverture du tube */ read(desc_tub, buf,
```

```
sizeof(buf) ); /* lecture dans le tube */ printf("%d -- j'ai reçu %s",getpid(),buf);}
```

La trace de ces deux programmes est :

```
$ ./red & ./lec
```

```
[1] 14906
```

```
14906--fin d'écriture 14907 -- j'ai reçu message
```

Chapitre 5 : - IPC par envoie de messages & Segment de mémoire partagée

Introduction

Les pipes permettent la communication entre processus, mais ils ont des limites :

- Les pipes anonymes fonctionnent uniquement entre processus liés (parent–enfant).
- Les pipes nommés sont plus flexibles, mais restent unidirectionnels et séquentiels.

Dans des systèmes plus complexes où plusieurs processus doivent échanger des données de façon dynamique et asynchrone, d'autres mécanismes d'IPC sont nécessaires :

- **Les files de messages** : permettent un échange structuré et persistant entre processus.
- **La mémoire partagée** : permet une communication très rapide en utilisant un espace mémoire commun

I/Compléments sur les Inter-Process Communication (IPC)

1. Inter-Process Communication (IPC)

Définition : Définition des IPC (Inter-Process Communication)

Les **IPC (Inter-Process Communication)** sont des mécanismes permettant aux processus sous Unix/Linux d'échanger des données et de partager des ressources.

Le **System V IPC** regroupe plusieurs outils historiques encore utilisés aujourd'hui.

Intérêt du System V IPC :

- Communication entre processus indépendants.
- Synchronisation entre processus concurrents.
- Gestion efficace de ressources partagées.

Types d'IPC System V :

1. **Files de messages** : msgget, msgsnd, msgrcv.
2. **Mémoire partagée** : shmget, shmat, shmdt, shmctl.
3. **Sémaphores** : semget, semop, semctl.

Gestion des IPC System V:

- Chaque ressource IPC est identifiée par une **clé unique** (`key_t`).
- Ces clés sont générées avec la fonction `ftok()`.

Generation d'une clé IPC avec `ftok()` :

- Sert à générer une clé IPC à partir d'un fichier existant et d'un identifiant.
- Syntaxe :
`key_t ftok(const char *pathname, int proj_id);`

Exemple : Exemple d'utilisation de ftok()

```
#include <stdio.h>

#include <sys/ipc.h>

int main() {

    key_t key;

    // Générer une clé IPC à partir du fichier /tmp et du caractère 'A'

    key = ftok("/tmp", 'A');

    if (key == -1) {

        perror("ftok");

        return 1;

    }

    printf("Clé IPC générée : %d\n", key);

    return 0;

}
```

Structure ipc_perm

La structure `ipc_perm` est utilisée dans les objets IPC System V (file de messages, mémoire partagée, sémaphores).

Elle stocke les **propriétaires**, **créateurs**, et les **droits d'accès**.

```
struct ipc_perm {

    uid_t uid; // UID du propriétaire

    gid_t gid; // GID du propriétaire

    uid_t cuid; // UID du créateur

    gid_t cgid; // GID du créateur

    mode_t mode; // Permissions (comme chmod)

};
```

Commandes Shell pour gérer les objets IPC

ipcs

Affiche les objets IPC du système : files de messages, sémaphores, mémoires partagées.

Exemple : `$ ipcs`

Affiche :

- **T** : type d'objet (**q** = msg queue, **m** = shared memory, **s** = semaphore)
- **ID** : identifiant interne
- **KEY** : clé IPC en hexadécimal
- **MODE** : droits d'accès
- **OWNER / GROUP** : propriétaire et groupe

\$ ipcs					
IPC status from /dev/kmem as of Tue Oct 20 08:56:30 1998					
T	ID	KEY	MODE	OWNER	GROUP
Message Queues:					
q	100	0x00000000	--rw-----	root	info
q	51	0x00000000	--rw-----	root	info
q	2	0x49179e95	--rw-rw-rw-	root	root
q	155	0x00000000	--rw-rw----	jmr	ens
Shared Memory:					
m	0	0x41440014	--rw-rw-rw-	root	root
m	1	0x41442041	--rw-rw-rw-	root	root
Semaphores:					
s	0	0x41442041	--ra-ra-ra-	root	root
s	1	0x4144314d	--ra-ra-ra-	root	root
\$					

Remarque :

Les objets IPC System V sont partagés grâce à une **clé externe** commune aux processus. Chaque objet possède :

- Une **clé** : joue le rôle d'un nom permettant d'identifier l'objet au niveau du système (analogue à un chemin de fichier).
- Un **identificateur interne (ID)** : équivalent au descripteur de fichier, utilisé par les processus pour manipuler l'objet.

Pour qu'un processus accède à un objet IPC, il doit obtenir son **ID** en utilisant sa **clé externe**. Les processus doivent donc partager cette clé et utiliser les primitives propres au type d'objet (file de messages, mémoire partagée, sémaphore).

2. Mode d'échange de données

Lorsqu'on parle de communication entre processus (IPC) ou d'échange de données en général, on distingue

1. Mode synchrone

- Émetteur et récepteur doivent être prêts en même temps.
- L'émetteur attend que le récepteur reçoive le message.
- Risque de blocage et ralentissement si l'un des deux n'est pas prêt.

2. Mode asynchrone

- L'émetteur envoie les données puis continue sans attendre.
- Le récepteur lit plus tard quand il est disponible.
- Plus flexible, évite les blocages et améliore la performance.

II/Communication par envoi de messages

La communication par envoi de messages consiste à transmettre explicitement des données entre processus. Sur une même machine, le système d'exploitation remet directement le message au processus cible. Sur des machines différentes, le message passe par un protocole réseau avant d'être délivré au destinataire. Le principe reste identique : un échange structuré de messages, local ou à distance.

1. Modes de Communication dans les Systèmes à Messages

Dans les systèmes à messages, le système d'exploitation assure la communication des processus à l'aide de deux

primitives: SEND et RECEIVE, selon deux types de communications directe et indirecte

1. Communication directe

- Les processus communiquent directement entre eux.
- Chaque processus doit connaître l'identité de l'autre.
- Primitives :
 - **SEND(B, msg)** : envoyer un message **msg** au processus B
 - **RECEIVE(A, msg)** : recevoir un message de A
- **Exemple** : Producteur-consommateur

`send(consommateur, elt);`

`receive(producteur, elt);`

- Chaque paire de processus établit automatiquement un lien direct. Le lien est symétrique : producteur et consommateur se connaissent mutuellement.

2. Communication indirecte

- Les messages passent par un objet intermédiaire appelé **Boîte-à-lettres** (mailbox).
- Deux processus ne communiquent que s'ils partagent la même boîte.
- Primitives :
 - **SEND(A, msg)** : envoyer un message vers la boîte A
 - **RECEIVE(A, msg)** : recevoir un message depuis la boîte A
- **Propriétés** :
 - Un lien existe seulement si une boîte est partagée
 - Une boîte peut être partagée par plusieurs processus
 - Plusieurs liens peuvent exister entre deux processus (via différentes boîtes)
 - Les liens peuvent être unidirectionnels ou bidirectionnels
- Une boîte-à-lettres peut fonctionner comme une **file de messages**, permettant un stockage temporaire et ordonné des messages.

2. File de message

La communication inter processus avec les files de message est un mécanisme permettant à des processus d'échanger des données sous forme de messages structurés à travers une implémentation du concept de boîte aux lettres.

2.1. File de messages

1. Définition

- Une file de messages est une structure de données gérée par le noyau permettant aux processus d'échanger des messages de manière asynchrone.
- Les messages restent dans la file jusqu'à ce qu'un processus les récupère.
- Permet le multiplexage : un message peut être lu par plusieurs processus.

2. Caractéristiques

- **Persistance** : La file existe indépendamment des processus.
- **Asynchronisme** : L'émetteur n'attend pas que le récepteur soit prêt.
- **Ordre** : Messages traités selon l'ordre d'arrivée, avec possibilité de priorité.

3. Méthode

- Chaque file est associée à une **clé** (équivalent numérique d'un nom).
- L'**identificateur de file** (**msgid**) est obtenu via cette clé.
- Les processus ayant connaissance de la file et les droits nécessaires peuvent :
 - Créer une file
 - Consulter ses caractéristiques :
 - Nombre de messages dans la file
 - Taille de la file
 - PID du dernier processus ayant écrit
 - PID du dernier processus ayant lu
 - Envoyer un message (**msgsnd**)
 - Lire un message (**msgrcv**)

4. Structure **msgid_ds**

Définie dans `<sys/msg.h>` :

```
struct msgid_ds {  
  
    struct ipc_perm msg_perm; // Permissions et propriétaire  
  
    time_t msg_stime;         // Dernier envoi (msgsnd)  
  
    time_t msg_rtime;         // Dernière réception (msgrcv)  
  
    time_t msg_ctime;         // Dernière modification des métadonnées (msgctl IPC_SET)  
  
    unsigned long msg_cbytes; // Nombre d'octets actuellement en file  
  
    msgqnum_t msg_qnum;       // Nombre actuel de messages en file  
  
    msglen_t msg_qbytes;      // Taille maximale en octets de la file  
  
    pid_t msg_lpid;           // PID du dernier processus ayant fait msgsnd  
  
    pid_t msg_lrpid;          // PID du dernier processus ayant fait msgrcv  
  
};
```

● Explication des champs :

- **msg_perm** : permissions et informations sur le propriétaire (utilisateur/groupe)
- **msg_stime** : timestamp du dernier envoi de message (**msgsnd**)
- **msg_rtime** : timestamp de la dernière réception (**msgrcv**)

- `msg_ctime` : timestamp du dernier changement des métadonnées (`msgctl IPC_SET`)
- `msg_cbytes` : octets actuellement présents dans la file
- `msg_qnum` : nombre de messages en attente
- `msg_qbytes` : capacité maximale de la file
- `msg_lspid` : PID du dernier processus ayant envoyé un message
- `msg_lrpid` : PID du dernier processus ayant reçu un message

2.2. Opérations sur les files de messages

2.2.1. Création et Accès à une file de message

1. Création et accès à une file de messages

- **Primitive utilisée : `msgget()`**

Permet de créer une file ou d'obtenir l'identifiant d'une file existante.

```
#include <sys/types.h>
```

```
#include <sys/ipc.h>
```

```
#include <sys/msg.h>
```

```
int msgget(key_t cle, int option);
```

- **Retour :**
 - Identifiant (`msgid`) de la file de messages associée à la clé
 - `-1` en cas d'erreur
- **Paramètres :**
 - `cle` : clé externe identifiant la file (`key_t`)
 - `option` : options de création et permissions
- **Remarque :** Tous les processus doivent partager une clé commune pour accéder à la même file.

2. Méthodes pour définir la clé IPC

1. **Clé fixée dans le code** : valeur codée en dur dans chaque programme.
2. **IPC_PRIVATE** : file accessible uniquement par le processus créateur et ses descendants.
3. **`ftok()`** : calcul automatique d'une clé unique à partir d'un fichier et d'un identifiant.

```
#include <sys/ipc.h>
```

```
key_t ftok(const char *ref, int numero);
```

- `ref` : chemin d'un fichier existant
- `numero` : entier pour différencier plusieurs files basées sur le même fichier

Exemple :

```
key_t cle = ftok("/rep/tp", 123);
```

```
int id = msgget(cle, IPC_EXCL | IPC_CREAT | 0660);
```

- **IPC_CREAT** : crée la file si elle n'existe pas
- **IPC_EXCL** : erreur si la file existe déjà
- **0660** : lecture/écriture pour propriétaire et groupe, aucun accès pour les autres

3. Structure des messages

- Chaque message contient :
 - **Type** (**long**) : permet de classer et filtrer les messages
 - **Contenu** (**char[]**) : données du message

```
struct msgbuf {
```

```
    long msgtype;    // Type du message
```

```
    char msgtext[long]; // Contenu du message
```

```
};
```

- **Multiplexage** : plusieurs types de messages peuvent coexister dans une même file, permettant:
 - Envoyer/recevoir différents types dans une seule file
 - Prioriser certains messages
 - Segmenter la file en catégories logiques

4. Fondamental

- La communication via une file de messages est **bidirectionnelle** : un consommateur peut aussi devenir producteur.
- Permet une communication efficace et asynchrone entre plusieurs processus.

2.2.2. Envoi d'un message

1. Primitive : **msgsnd()**

Permet d'envoyer un message dans une file de messages System V.

```
#include <sys/types.h>
```

```
#include <sys/ipc.h>
```

```
#include <sys/msg.h>
```

```
int msgsnd(int msg_id, void *pt_msg, int size, int option);
```

- **Retour** :

- Taille du message envoyé
- -1 en cas d'erreur
- **Paramètres :**
 - `msg_id` : identifiant de la file de messages (retourné par `msgget()`)
 - `pt_msg` : pointeur vers la structure `msgbuf` contenant le message
 - `size` : taille du corps du message (`sizeof()` ou `strlen()`)
 - `option` : contrôle l'envoi du message

2. Options possibles pour `msgsnd()`

Option	Description
0	Comportement par défaut : bloquant si la file est pleine, attend qu'une place se libère
IPC_NOWAIT	Envoi non bloquant : échoue immédiatement avec erreur <code>EAGAIN</code> si la file est pleine
MSG_NOERROR	Troncature du message si sa taille dépasse la capacité de la file
Combinaison	On peut combiner plusieurs options avec `

Exemple de combinaison :

```
msgsnd(msg_id, &msg, sizeof(msg.mtext), IPC_NOWAIT | MSG_NOERROR);
```

3. Structure d'un message

```
struct msgbuf {
    long mtype;    // Type du message
    char mtext[100]; // Corps du message
};
```

- `mtype` : type ou catégorie du message pour le multiplexage
- `mtext` : données du message

4. Exemple complet d'envoi

```
#include <stdio.h>

#include <sys/types.h>

#include <sys/ipc.h>

#include <sys/msg.h>

#include <string.h>

struct msgbuf {

    long mtype;

    char mtext[100];

};

int main() {

    int msg_id;

    struct msgbuf msg;

    // Création ou récupération de la file de messages

    msg_id = msgget(1234, IPC_CREAT | 0666);

    if (msg_id == -1) {

        perror("msgget");

        return 1;

    }

    // Remplir le message

    msg.mtype = 1;

    strcpy(msg.mtext, "Bonjour, ceci est un test");

    // Envoyer le message en mode non bloquant

    if (msgsnd(msg_id, &msg, sizeof(msg.mtext), IPC_NOWAIT) == -1) {

        perror("msgsnd");

        return 1;

    }

}
```

```

printf("Message envoyé avec succès !\n");

return 0;

}

```

- Ici, le message est envoyé **sans bloquer** (**IPC_NOWAIT**) et peut être tronqué si nécessaire (**MSG_NOERROR** si ajouté).

2.2.3. Réception d'un message

1. Primitive : **msgrecv()**

Permet à un processus de lire un message depuis une file de messages System V.

```
#include <sys/types.h>
```

```
#include <sys/ipc.h>
```

```
#include <sys/msg.h>
```

```
int msgrecv(int msg_id, void *pt_msg, int size, long msgtype, int option);
```

- **Retour :**
 - Taille en octets du message reçu
 - **-1** en cas d'erreur
- **Paramètres :**
 - **msg_id** : identifiant de la file (retourné par **msgget()**)
 - **pt_msg** : pointeur vers la structure **msgbuf** pour stocker le message reçu
 - **size** : taille du corps du message (sans **mttype**)
 - **msgtype** : définit quel message lire
 - **>0** : premier message dont **mttype == msgtype**
 - **0** : premier message disponible
 - **<0** : message dont **mttype** est le plus petit $\geq |\text{msgtype}|$
 - **option** : contrôle le comportement de réception

2. Options possibles pour **msgrecv()**

Option	Description
0	Mode par défaut : bloquant jusqu'à ce qu'un message correspondant soit disponible
IPC_NOWAIT	Non bloquant : échoue immédiatement si aucun message disponible

MSG_NOERROR	Tronque le message si sa taille dépasse size sans générer d'erreur
MSG_EXCEPT	Utilisé avec msgtype>0 , récupère le premier message différent de msgtype

Exemple de combinaison :

```
msgrcv(msg_id, &msg, sizeof(msg.mtext), 1, IPC_NOWAIT | MSG_NOERROR);
```

- Ne bloque pas, tronque si nécessaire, lit uniquement les messages de type 1

3. Gestion des priorités avec **msgtype**

Si trois types existent :

- **type=1** : messages prioritaires
- **type=2** : priorité moyenne
- **type=3** : moins prioritaires

Exemples :

```
msgrcv(..., 1, ...) // reçoit un message de priorité max
```

```
msgrcv(..., -2, ...) // reçoit message priorité 1 si dispo, sinon priorité 2
```

```
msgrcv(..., -3, ...) // reçoit priorité 1, sinon 2, sinon 3
```

4. Structure du message

```
struct msgbuf {
    long mtype;    // Type du message
    char mtext[100]; // Contenu du message
};
```

- **mtype** : permet le multiplexage
- **mtext** : données du message

5. Exemple complet de réception d'un message

```
#include <stdio.h>
```

```
#include <sys/types.h>
```

```
#include <sys/ipc.h>
```

```

#include <sys/msg.h>

#include <string.h>

struct msgbuf {

    long mtype;    // Type du message

    char mtext[100]; // Corps du message

};

int main() {

    int msg_id;

    struct msgbuf msg;

    // Récupération de la file de messages existante

    msg_id = msgget(1234, 0666);

    if (msg_id == -1) {

        perror("msgget");

        return 1;

    }

    // Réception d'un message de type 1 (bloquant)

    if (msgrcv(msg_id, &msg, sizeof(msg.mtext), 1, 0) == -1) {

        perror("msgrcv");

        return 1;

    }

    printf("Message reçu : %s\n", msg.mtext);

    return 0;

}

```

- Ici, le message est **lu de manière bloquante** et de type 1.

2.2.4. La primitive `msgctl`

1. Fonction : **msgctl()**

Permet de contrôler, consulter, modifier ou supprimer une file de messages System V.

```
#include <sys/types.h>
```

```
#include <sys/ipc.h>
```

```
#include <sys/msg.h>
```

```
int msgctl(int msg_id, int command, struct msqid_ds *buf);
```

- **Paramètres :**

- **msg_id** : identifiant de la file (**msgget()**)
- **command** : opération à effectuer (**IPC_STAT**, **IPC_SET**, **IPC_RMID**)
- **buf** : pointeur vers une structure **msqid_ds** pour lire ou modifier les infos

- **Retour :**

- **0** si succès
- **-1** en cas d'erreur

2. Commandes disponibles

Commande	Description
IPC_STAT	Remplit buf avec les informations actuelles de la file
IPC_SET	Modifie certains paramètres (permissions, UID/GID) selon buf
IPC_RMID	Supprime la file et tous ses messages

3. Exemples d'utilisation

a) Obtenir les informations d'une file

```
struct msqid_ds info;
```

```
msgctl(msg_id, IPC_STAT, &info);
```

```
printf("Nombre de messages : %lu\n", info.msg_qnum);
```

- Remplit **info** avec les infos de la file, puis affiche le nombre de messages.

b) Modifier les permissions de la file

```
struct msqid_ds info;
```

```
msgctl(msg_id, IPC_STAT, &info); // Récupérer les infos actuelles  
info.msg_perm.mode = 0660;      // Modifier permissions (lecture/écriture pour owner & groupe)  
msgctl(msg_id, IPC_SET, &info); // Appliquer les changements
```

- Change les permissions de la file en lecture/écriture pour le propriétaire et le groupe.

c) Supprimer la file de messages

```
msgctl(msg_id, IPC_RMID, NULL);  
printf("File de messages supprimée !\n");
```

- Supprime la file et tous ses messages restants. Après suppression, `msg_id` devient invalide.

2.2.5. Destruction d'une file de message

1. Destruction via la primitive `msgctl()`

- La file de messages est détruite en utilisant `msgctl()` avec l'opération `IPC_RMID`.
- **Prototype :**

```
#include <sys/types.h>  
#include <sys/ipc.h>  
#include <sys/msg.h>
```

```
msgctl(int msg_id, IPC_RMID, NULL);
```

- **Retour :**
 - 0 si succès
 - -1 en cas d'erreur

Exemple en C :

```
msgctl(msg_id, IPC_RMID, NULL);  
printf("File de messages supprimée !\n");
```

- Supprime la file et tous les messages restants. Après suppression, `msg_id` devient invalide.

2. Destruction depuis le shell

- Avec l'identifiant interne de la file : `ipcrm -q identifiant`
- Avec la clé externe de la file : `ipcrm -Q cle`
- Ces commandes permettent de supprimer une file de messages directement depuis le terminal.

2.2.6. Exemple d'utilisation d'une File de messages

Programme C : File de messages

```
#include <stdio.h>

#include <stdlib.h>

#include <string.h>

#include <sys/types.h>

#include <sys/ipc.h>

#include <sys/msg.h>

#define CLE 12

struct message {

    long letype;

    char *chaine; // ou char chaine[30];

};

int main() {

    int msqid;

    struct message le_message;

    // Création de la file de messages

    msqid = msgget(CLE, IPC_CREAT | IPC_EXCL | 0777);

    le_message.letype = 0; // type du message

    le_message.chaine = "hello, je suis le processus fils\0";

    if (fork() == 0) { // Processus fils

        printf("je suis le fils j'envoie un message pour mon père\n");

        msgsnd(msqid, &le_message, sizeof(le_message.chaine), 0); // Envoi du message

        exit(0);

    }

    // Processus père : réception du message

    msgrcv(msqid, &le_message, sizeof(le_message.chaine), 0, 0);
```

```

printf("je suis le père et j'ai reçu le message de mon fils : %s\n", le_message.chaine);

// Destruction de la file de messages

msgctl(msqid, IPC_RMID, NULL);

return 0;

}

```

Trace du programme

je suis le fils j'envoie un message pour mon père

je suis le père et j'ai reçu le message de mon fils : hello, je suis le processus fils

Explication :

- Le processus fils envoie un message via la file de messages.
- Le père reçoit le message et l'affiche.
- La file de messages est ensuite détruite avec `msgctl(..., IPC_RMID, NULL)`.

III/Communication par segment de mémoire partagée

Lors de l'échange de volumes importants de données (fichier, tube ou file de messages), il faut recopier les informations échangées dans un tampon système ; d'où un passage en mode noyau du processus à l'origine de cet échange.

1. Définition

La **mémoire partagée** est un segment de mémoire accessible par plusieurs processus.

- Contrairement aux pipes ou files de messages, elle permet un **accès direct aux données**, sans copie dans un tampon système, ce qui réduit le coût en temps et en ressources.
- Les échanges se font en **lisant ou écrivant directement** dans l'espace mémoire commun.

Accès au segment partagé :

- Tous les **processus fils** du créateur.
- **Processus non liés**, si les droits d'accès le permettent.

Attention :

- L'exclusion mutuelle n'est pas automatique ; il faut gérer l'accès concurrent (sémaphores, mutex).

2. Avantages et Inconvénients

Avantages :

- Pas de duplication des données → **efficacité mémoire**
- Communication rapide entre processus
- Idéal pour partager des **grandes structures de données**

Inconvénients :

- Pas de protection contre les accès concurrents
- Le segment doit être **supprimé après utilisation** pour éviter les fuites mémoire

2. Création et Gestion d'un Segment de Mémoire Partagée

1. Principes de fonctionnement

L'utilisation de la mémoire partagée suit ces étapes :

1. **Création** du segment (ou attachement à un segment existant)
2. **Attachement** du segment à l'espace mémoire du processus (`shmat()`)
3. **Lecture et écriture** des données
4. **Détachement** du segment (`shmdt()`) lorsqu'il n'est plus utilisé
5. **Destruction** du segment (`shmctl()`) lorsqu'aucun processus ne l'utilise

System V IPC fournit 4 appels principaux pour gérer la mémoire partagée :

- `shmget()` : Crée ou récupère un segment
- `shmat()` : Attache un segment au processus
- `shmdt()` : Détache un segment du processus
- `shmctl()` : Supprime ou gère un segment

2. Création et récupération d'un segment

Prototype de `shmget()` :

```
#include <sys/types.h>
```

```
#include <sys/ipc.h>
```

```
#include <sys/shm.h>
```

```
int shmget(key_t cle, int taille, int option);
```

Retour :

- Identifiant du segment (utilisé pour les autres opérations)
- `-1` en cas d'erreur (`errno` définit l'erreur)

Paramètres :

- `key_t cle` : clé IPC pour identifier le segment
 - `IPC_PRIVATE` : segment accessible uniquement par le processus créateur et ses descendants

- `ftok()` : permet de générer une clé unique partagée par plusieurs processus
- `int taille` : taille en octets du segment (ex. `sizeof(struct nom)` ou valeur fixe)
- `int option` : combinaison des constantes
 - `IPC_CREAT` : crée le segment si inexistant
 - `IPC_EXCL` : erreur si le segment existe déjà
 - Permissions d'accès (ex. `0660` pour lecture/écriture propriétaire et groupe)

Exemple :

```
int id = shmget(IPC_PRIVATE, 1024, IPC_EXCL | IPC_CREAT | 0660);
```

- Crée un segment de **1024 octets**
- **Lecture/écriture** pour le propriétaire et son groupe
- Interdit la réutilisation d'un segment existant

Principaux codes d'erreur (`errno`) :

- `EEXIST` : segment existe déjà avec `IPC_EXCL`
- `EACCES` : permission refusée
- `EINVAL` : taille invalide ou combinaison d'options incorrecte
- `ENOMEM` : mémoire insuffisante

Code	Constante	Signification
1	<code>EPERM</code>	Opération non permise (permissions insuffisantes).
2	<code>ENOENT</code>	Fichier ou segment IPC inexistant. □
12	<code>ENOMEM</code>	Pas assez de mémoire disponible. □
13	<code>EACCES</code>	Accès refusé (permissions insuffisantes). □
17	<code>EEXIST</code>	Ressource IPC existe déjà.
28	<code>ENOSPC</code>	Pas assez d'espace pour créer un segment IPC.
22	<code>EINVAL</code>	Argument invalide (ex : taille incorrecte pour <code>shmget</code>).

2.3. Accès à un segment de mémoire partagée

1. Attachement d'un segment

Pour utiliser un segment de mémoire partagée, un processus doit **l'attacher** à son espace mémoire avec la primitive `shmat()`.

Prototype :

```
#include <sys/types.h>
```

```
#include <sys/ipc.h>
```

```
#include <sys/shm.h>
```

```
void *shmat(int shm_id, const void *shm_addr, int option);
```

Retour :

- **Succès** : adresse du segment en mémoire
- **Échec** : `(void *) -1` avec `errno` défini (`EINVAL`, `EIDRM`, etc.)

Paramètres :

- `shm_id` : identifiant du segment obtenu via `shmget()`
- `shm_addr` : adresse où attacher le segment (mettre `NULL` pour laisser le système choisir)
- `option` : indique les droits d'accès :
 - `SHM_R` : lecture seule
 - `SHM_W` : écriture seule
 - `SHM_R | SHM_W` : lecture et écriture

2.4. Détachement (Libération) d'un segment de mémoire partagée

Libération d'un segment

Quand un processus a terminé son travail sur un segment de mémoire partagée, il doit **le détacher** pour indiquer au système qu'il ne l'utilise plus, avec la primitive `shmdt()`.

Prototype :

```
#include <sys/types.h>
```

```
#include <sys/ipc.h>
```

```
#include <sys/shm.h>
```

```
int shmdt(const void *shm_addr);
```

Principe :

- `shm_addr` : adresse du segment renvoyée par `shmat()`
- Après détachement, le processus **ne peut plus accéder** au segment.
- Le segment **n'est pas détruit**, il reste accessible aux autres processus attachés.
- Si le processus oublie de détacher le segment, le système le fera **automatiquement à sa mort**

2.5. Contrôle d'un segment de mémoire partagée

Contrôle d'un segment de mémoire partagée

1. Fonction `shmctl()`

La primitive `shmctl()` permet de contrôler un segment de mémoire partagée : obtenir des informations, modifier les permissions ou le supprimer.

Prototype :

```
#include <sys/types.h>
```

```
#include <sys/ipc.h>
```

```
#include <sys/shm.h>
```

```
int shmctl(int shm_id, int cmd, struct shmid_ds *desc);
```

Paramètres :

- **shm_id** : identifiant du segment (retourné par **shmget**)
- **cmd** : commande à exécuter :
 - **IPC_STAT** : récupère les informations du segment dans **shm_ds**
 - **IPC_SET** : modifie les paramètres du segment selon **shm_ds**
 - **IPC_RMID** : supprime le segment
- **desc** : pointeur vers une structure **shm_ds** utilisée avec **IPC_STAT** ou **IPC_SET**.

Retour :

- 0 en cas de succès
- -1 en cas d'erreur (**errno** défini)

2. Structure **shm_ds**

Contient les informations d'un segment :

```
struct shm_ds {  
  
    struct ipc_perm shm_perm; // Droits d'accès  
  
    size_t shm_segsz;        // Taille du segment en octets  
  
    pid_t shm_lpid;          // PID du dernier processus attaché  
  
    pid_t shm_cpid;          // PID du créateur  
  
    shmatt_t shm_nattch;     // Nombre de processus attachés  
  
    time_t shm_atime;        // Dernier accès  
  
    time_t shm_dtime;        // Dernier détachement  
  
    time_t shm_ctime;        // Dernière modification  
  
};
```

Remarque :

- Si **cmd = IPC_RMID**, le segment sera détruit après le **dernier détachement** (**shm_nattch == 0**).

3. Exemple : Suppression d'un segment

```
#include <stdio.h>  
  
#include <stdlib.h>  
  
#include <sys/ipc.h>
```

```

#include <sys/shm.h>

#include <sys/types.h>

int main() {

    key_t key = ftok("shmfile", 65);

    int shmid = shmget(key, 1024, 0666 | IPC_CREAT);

    if (shmid == -1) {

        perror("shmget");

        exit(1);

    }

    // Suppression du segment

    if (shmctl(shmid, IPC_RMID, NULL) == -1) {

        perror("shmctl");

        exit(1);

    }

    printf("Segment de mémoire partagée supprimé.\n");

    return 0;

}

```

- Ici, un segment de mémoire partagée est créé puis **supprimé immédiatement** via **IPC_RMID**.

Exemple:

```

#include <stdio.h>

#include <stdlib.h>

#include <sys/ipc.h>

#include <sys/shm.h>

#include <sys/types.h>

int main() {

    // Générer une clé IPC

```

```

key_t key = ftok("shmfile", 65);

if (key == -1) {
    perror("ftok");
    exit(1);
}

// Créer ou récupérer le segment de mémoire partagée
int shmid = shmget(key, 1024, 0666 | IPC_CREAT);

if (shmid == -1) {
    perror("shmget");
    exit(1);
}

// Déclarer la structure pour stocker les infos
struct shmid_ds desc;

// Récupérer les informations du segment
if (shmctl(shmid, IPC_STAT, &desc) == -1) {
    perror("shmctl");
    exit(1);
}

// Afficher les informations
printf("Taille du segment : %zu octets\n", desc.shm_segsz);
printf("PID du créateur : %d\n", desc.shm_cpid);
printf("Nombre de processus attachés : %ld\n", desc.shm_nattch);

return 0;
}

```

2.6. Exemple de communication entre deux processus par l'intermédiaire d'un segment de

Exemple : Communication entre deux processus avec mémoire partagée

Description :

Un processus fils écrit dans un segment de mémoire partagée, et le processus père lit ce contenu.

Code corrigé :

```
#include <stdio.h>

#include <stdlib.h>

#include <string.h>

#include <sys/types.h>

#include <sys/ipc.h>

#include <sys/shm.h>

#include <sys/wait.h>

#define TAILLE_SHM 1024

int main(int argc, char *argv[]) {

    key_t key;

    int shmid;

    char *data;

    // Création de la clé IPC

    key = ftok("exo.c", 'R');

    if (key == -1) {

        perror("ftok");

        exit(1);

    }

    // Création du segment de mémoire partagée

    shmid = shmget(key, TAILLE_SHM, IPC_CREAT | 0644);

    if (shmid == -1) {

        perror("shmget");

        exit(1);    }

    if (fork() == 0) { // Processus fils
```

```

data = shmat(shmid, NULL, SHM_W);

if (data == (void *)-1) {

    perror("shmat fils");

    exit(1);

}

if (argc == 2) {

    printf("Écriture de \"%s\" en mémoire partagée\n", argv[1]);

    strncpy(data, argv[1], TAILLE_SHM);

}

exit(0);

} else { // Processus père

    wait(NULL); // Attendre la fin du fils

    data = shmat(shmid, NULL, SHM_R);

    if (data == (void *)-1) {

        perror("shmat père");

        exit(1);

    }

    printf("Contenu de la mémoire partagée : \"%s\"\n", data);

}

return 0;

}

```

Trace d'exécution :

```
$ ./a.out "section_systèmes informatiques"
```

Écriture de "section_systèmes informatiques" en mémoire partagée

Contenu de la mémoire partagée : "section_systèmes informatiques"

Points clés :

- `shmget()` : création du segment.
- `shmat()` : attachement du segment à chaque processus.
- `fork()` : crée un processus fils qui écrit.
- `wait()` : permet au père d'attendre le fils avant lecture.
- `strncpy()` : copie sécurisée des données.
- Communication bidirectionnelle possible via le même segment si nécessaire.

2.7. Problèmes liés à l'absence de synchronisation

Problèmes principaux :

1. Conditions de course (Race Conditions)

- Plusieurs processus écrivent simultanément sur un segment partagé.
- L'ordre d'exécution étant non déterministe, les données peuvent être corrompues.
- **Exemple :**
 - Processus A écrit "HELLO"
 - Processus B écrit "WORLD"
 - Résultat dans le segment : "HELOD" (incohérent)

2. Lecture partielle de données

- Si un processus lit pendant qu'un autre écrit, il peut récupérer des données incomplètes ou incorrectes.

3. Écrasement des données

- Un processus peut écraser les données d'un autre sans avertissement.
- **Exemple de corruption :** deux processus incrémentent un compteur sans protection.
 - Résultat attendu : 2 000 000
 - Résultat réel : valeur incorrecte à cause des accès concurrents.

Exemple de corruption sans synchronisation

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <sys/ipc.h>
```

```
#include <sys/shm.h>
```

```
#include <sys/types.h>
```

```
#include <unistd.h>
```

```
#include <wait.h>
```

```
#define SHM_SIZE sizeof(int) // Taille du segment de mémoire partagée
```

```

int main() {

    key_t key = ftok("shmfile", 65); // Génération de la clé IPC

    int shmid = shmget(key, SHM_SIZE, 0666 | IPC_CREAT); // Création du segment

    int *compteur = (int *) shmat(shmid, NULL, 0); // Attachement

    *compteur = 0; // Initialisation

    pid_t pid = fork(); // Création du processus fils

    if (pid == 0) { // Processus fils

        for (int i = 0; i < 1000000; i++)

            (*compteur)++; // Incrémentation concurrente

        shmdt(compteur); // Détachement

        exit(0);

    } else { // Processus père

        for (int i = 0; i < 1000000; i++)

            (*compteur)++; // Incrémentation concurrente

        wait(NULL); // Attente de la fin du fils

        printf("Valeur finale du compteur : %d\n", *compteur); // Affichage

        shmdt(compteur); // Détachement

        shmctl(shmid, IPC_RMID, NULL); // Suppression

    }

    return 0;

}

```

Trace possible :

Valeur finale du compteur : 1427658

Explication :

- Chaque processus incrémente 1 000 000 fois.
- Sans synchronisation, plusieurs accès simultanés écrasent des valeurs.
- **Exemple :**

- Processus 1 lit **compteur = 100**
- Processus 2 lit aussi **compteur = 100**
- Les deux stockent **101** → perte d'une mise à jour
- Résultat final incorrect.

Correction :

- Utilisation d'un **mutex** pour protéger l'accès concurrent au compteur.

Conclusion sur la mémoire partagée :

- La mémoire partagée est un outil puissant pour la communication rapide entre processus, mais elle devient risquée sans mécanismes de synchronisation.
- **Cas où la synchronisation n'est pas nécessaire :**
 - Un seul écrivain et plusieurs lecteurs.
 - Un seul processus accède à la mémoire à la fois.
- **Cas nécessitant une synchronisation :**
 - Plusieurs écrivains concurrents.
 - Processus lisant et écrivant en parallèle.

MAGUEMOUN SAMY

Chapitre 6 - Les Signaux en Communication Inter-Processus (IPC)

I/Les Signaux en Communication Inter- Processus (IPC)

Définition :

Les signaux sont des interruptions logicielles permettant de notifier un processus qu'un événement particulier s'est produit. Ils servent à :

- Gérer des interruptions,
- Synchroniser des processus,
- Gérer des erreurs système.

Ils permettent une communication **asynchrone**, mais avec une quantité d'information limitée.

Caractéristiques :

- Notification envoyée par le noyau ou un autre processus.
- Ne transporte pas d'information supplémentaire (sauf **sigqueue**).
- Peut être capturé, ignoré ou traité par défaut.
- Chaque signal possède :
 - Un nom constant commençant par **SIG**,
 - Un numéro de 1 à **NSIG**,
 - Un gestionnaire (handler) associé.
- La liste des signaux sous Linux : **kill -l**.

Utilisation :

- Terminaison : **SIGTERM**, **SIGKILL**
- Synchronisation : **SIGUSR1**, **SIGUSR2**
- Détection d'erreurs : **SIGSEGV**, **SIGBUS**
- Interruptions clavier : **SIGINT**
- Rechargement configuration : **SIGHUP**

La liste complète des signaux définis par le système linux

La liste complète des signaux définis par le système linux est donnée par la commande Shell **kill -l**

1. Signaux principaux et traitements par défaut :

- 1 : terminaison de processus
- 2 : terminaison avec création de core dump
- 3 : signal ignoré
- 4 : suspension du processus
- 5 : continuation d'un processus stoppé

```
~$ kill -l
1) SIGHUP      2) SIGINT      3) SIGQUIT     4) SIGILL      5) SIGTRAP
6) SIGABRT     7) SIGBUS      8) SIGFPE      9) SIGKILL     10) SIGUSR1
11) SIGSEGV    12) SIGUSR2    13) SIGPIPE    14) SIGALRM     15) SIGTERM
16) SIGSTKFLT  17) SIGCHLD    18) SIGCONT    19) SIGSTOP     20) SIGTSTP
21) SIGTTIN    22) SIGTTOU    23) SIGURG     24) SIGXCPU     25) SIGXFSZ
26) SIGVTALRM  27) SIGPROF    28) SIGWINCH   29) SIGIO       30) SIGPWR
31) SIGSYS     34) SIGRTMIN   35) SIGRTMIN+1 36) SIGRTMIN+2 37) SIGRTMIN+3
38) SIGRTMIN+4 39) SIGRTMIN+5 40) SIGRTMIN+6 41) SIGRTMIN+7 42) SIGRTMIN+8
43) SIGRTMIN+9 44) SIGRTMIN+10 45) SIGRTMIN+11 46) SIGRTMIN+12 47) SIGRTMIN+13
48) SIGRTMIN+14 49) SIGRTMIN+15 50) SIGRTMAX-14 51) SIGRTMAX-13 52) SIGRTMAX-12
53) SIGRTMAX-11 54) SIGRTMAX-10 55) SIGRTMAX-9  56) SIGRTMAX-8  57) SIGRTMAX-7
58) SIGRTMAX-6  59) SIGRTMAX-5 60) SIGRTMAX-4 61) SIGRTMAX-3 62) SIGRTMAX-2
63) SIGRTMAX-1  64) SIGRTMAX
```

Signal	Numéro	Description	Traitement par défaut
SIGHUP	1	Terminal déconnecté	(1)
SIGINT	2	Interruption (CTRL-C) émise à tous les processus associés à un terminal.	(1)
SIGQUIT	3	Arrête le processus (Ctrl + \) et génère un fichier core dump	(2)
SIGILL	4	Instruction illégale	(2)
SIGABRT	6	Abandon (abort())	(1)
SIGFPE	8	Erreur arithmétique (division par zéro, ...)	(1)
SIGKILL	9	Destruction immédiate du processus	(1) * non modifiable
SIGSEGV	11	Erreur d'accès mémoire	(2)
SIGPIPE	13	Écriture sur un pipe fermé	(1)
SIGALRM	14	Minuterie expirée (alarm())	(1)
SIGTERM	15	Demande de terminaison	(1)
SIGUSR1	10	Signal Utilisateur 1: assure la communication interprocessus.	(1)
SIGUSR2	12	Signal utilisateur 2	(1)
SIGCHLD	17	Envoyé au processus père a la terminaison d'un processus fils.	(3)
SIGSTOP	19	Arrêt du processus (inarrêtable)	(4) *
SIGTSTP	20	Pause via Ctrl+Z	(4)
SIGCONT	18	Reprise d'un processus interrompu	(5) *

Remarque : Le signal 0 sert uniquement à vérifier l'existence d'un processus.

2. Gestion des signaux

1. Émetteurs d'un signal :

- **Système d'exploitation** : informe d'événements anormaux, par exemple :
 - Erreur de virgule flottante (division par zéro)
 - Violation mémoire (segmentation fault)
- **Processus utilisateur** : pour coordination entre processus.
- **Utilisateur via clavier** :
 - CTRL + C → SIGINT (interruption)
 - CTRL + Z → SIGTSTP (suspension)

2. Récepteurs d'un signal :

- **Qui peut recevoir** :
 - Tout processus utilisateur (arrêt, suspension, manipulation)
 - Processus en arrière-plan ou premier plan
 - Processus fils (parent → enfants)
 - Démons (daemons) via `systemctl` ou `kill`
- **Qui ne peut pas recevoir** :
 - Noyau (PID 0)
 - Processus système protégés (ex. `init`, `systemd`)

3. États d'un signal :

- **En attente (Pending)** : signal non encore traité
- **Délivré (Delivered)** : signal reçu et exécuté

4. Prise en compte d'un signal :

- Exécution du **handler** associé :
 - Par défaut (`SIG_DFL`) : action standard du système
 - Ignoré (`SIG_IGN`) : signal ignoré
 - Handler personnalisé : exécute une routine spécifique
- Signaux spéciaux :

- **SIGKILL** : tue le processus
 - **SIGSTOP** : stoppe le processus (peut être repris)
 - **SIGCONT** : reprend un processus stoppé
 - Ces signaux spéciaux **ne peuvent pas** être interceptés, bloqués ou ignorés.
5. **Traitements par défaut des signaux :**
- Chaque signal a un handler par défaut (**SIG_DFL**) avec action possible :
 - **A (abort)** : arrêt du processus
 - **C (continue)** : reprise si suspendu, sinon ignoré
 - **S (stop)** : arrêt temporaire du processus
 - **E (exit)** : exécution de la procédure de service obligatoire
 - **I (ignore)** : ignorer le signal
 - **F** : le signal ne peut pas être ignoré

Exemples concrets :

- **SIGINT** → par défaut interrompt le processus (CTRL+C)
- **SIGKILL** → arrête immédiatement le processus
- **SIGSTOP** → stoppe le processus (CTRL+Z), à reprendre avec **SIGCONT**

Signal	Num	Type	Action par défaut (SIG_DFL)	Description
SIGKILL	9	SEF	A (Abort)	Termine immédiatement l'exécution du processus (ne peut pas être intercepté ou ignoré).
SIGSTOP	19	SEF	S (Stop)	Suspend immédiatement le processus (ne peut pas être intercepté ou ignoré).
SIGCONT	18	CEF	C (Continue)	Reprend l'exécution d'un processus suspendu.
SIGCHLD	17	I	I (Ignore)	Indique qu'un processus fils s'est terminé ou a changé d'état.
SIGUSR1	10	A	E (Exit)	Signal défini par l'utilisateur, peut être utilisé pour des traitements personnalisés.
SIGUSR2	12	A	E (Exit)	Signal défini par l'utilisateur, similaire à SIGUSR1.

3. Manipulation des signaux

La manipulation des signaux peut se faire :

- **Par ligne de commande**
- **Dans un programme utilisateur** via :
 - **kill()** : envoyer un signal à un processus
 - **signal()** : définir un handler personnalisé
 - **pause()** : mettre un processus en attente d'un signal

3.1. Envoyer un signal avec **kill()**

- **Prototype :**

```
#include <sys/types.h>
```

```
#include <signal.h>
```

```
int kill(pid_t pid, int signal);
```

- **Fonction** : envoie un signal à un processus identifié par son **pid**.
- **Retour** : 0 si succès, -1 sinon.
- **Valeurs possibles pour **pid**** :
 - **pid > 0** : envoie le signal au processus identifié par **pid**
 - **pid = 0** : envoie le signal à tous les processus du même groupe que le processus appelant

- `pid = -1` : envoie le signal à tous les processus sauf `init`
- `pid < -1` : envoie le signal à tous les processus du groupe `-pid`

Exemple : Envoyer SIGKILL à tous les processus d'un groupe

```
kill(-1234, SIGKILL); // Tue tous les processus du groupe 1234
```

Attention :

- Si `signal = 0`, aucun signal n'est envoyé. La valeur de retour sert à tester l'existence du processus.

Exemple : Vérifier l'existence d'un processus père

```
#include <stdio.h>

#include <unistd.h>

#include <signal.h>

#include <stdlib.h>

int main() {

    int pid;

    pid = fork();

    if (pid > 0) {

        if (kill(pid, 0) == 0)

            printf("le processus de pid=%d existe\n", pid);

        else

            printf("le processus de pid=%d n'existe pas\n", pid);

    }
}
```

Trace :

```
$ le processus de pid=3038 existe
```

Envoi depuis la ligne de commande :

- Syntaxe :

```
kill [option] <PID>
```

```
kill -s <signal> <PID>
```

kill -<signal> <PID>

- **<PID>** : identifiant du processus
- **<signal>** : nom ou numéro du signal
- Option **-s** : spécifier le signal par son nom plutôt que son numéro

3.2. Intercepter un signal avec la primitive signal()

1. Avec la primitive **signal()**

- Les signaux (sauf **SIGKILL**, **SIGCONT**, **SIGSTOP**) peuvent être interceptés.
- Permet d'installer un **handler spécifique** pour un signal donné.
- **Prototype :**

```
#include <signal.h>
```

```
sighandler_t signal(int sig, sighandler_t new_handler);
```

- **new_handler** : fonction exécutée à la réception du signal **sig**.
- Retour : **new_handler** ou **SIG_ERR** en cas d'erreur.
- Fonctionnement :
 - Le handler reçoit le numéro du signal.
 - À la fin du handler, le processus reprend son exécution là où il était suspendu.

2. Avec le shell : commande **trap**

- Permet de définir un handler pour un ou plusieurs signaux directement depuis le shell.
- **Syntaxe** : `trap 'new_handler' signal1 signal2 ...`

new_handler : commande ou script à exécuter lors de la réception du signal.

signal1, signal2, ... : noms ou numéros des signaux à intercepter.

Exemple :

```
:~$ trap "echo le signal SIGINT est reçu" 2
```

```
:~$ ^C
```

le signal SIGINT est reçu

- Pour désactiver le trap :

```
trap - 2
```

Résumé :

- **signal()** : interception d'un signal via un programme C.
- **trap** : interception d'un signal depuis le shell.
- Permet au processus ou au script de réagir à un signal reçu.

3.3. Attente d'un signal avec la primitive pause()

1. Fonction `pause()`

- **Prototype :**

```
#include <unistd.h>
```

```
int pause(void);
```

- Bloque le processus appelant jusqu'à la réception d'un signal.
- Retourne toujours `-1` avec `errno = EINTR` lorsqu'un signal est reçu.
- À la réception du signal :
 - Le processus peut se terminer (handler = `SIG_DFL`).
 - Passer à l'état stoppé (`SIGSTOP`).
 - Réexécuter `pause()` si réveillé par `SIGCONT` ou `SIGTERM` avec handler `SIG_IGN`.
 - Exécuter le handler associé au signal.
- Limitation : `pause()` ne permet pas de filtrer le type de signal reçu ni de connaître son nom.

2. Exemple pratique : communication fils → père via signal

```
#include <stdio.h>
```

```
#include <signal.h>
```

```
#include <unistd.h>
```

```
#include <stdlib.h>
```

```
int nb_req = 0;
```

```
int pid_fils, pid_pere;
```

```
void Hand_Pere(int sig){
```

```
    nb_req++;
```

```
    printf("\t le pere traite la requête numero %d du fils \n", nb_req);
```

```
}
```

```
int main() {
```

```
    if ((pid_fils = fork()) == 0) { /* FILS */
```

```
        pid_pere = getppid();
```

```
        sleep(2); /* laisser le temps au pere de se mettre en pause */
```

```
        for (int i = 0; i < 10; i++) {
```

```
            printf("le fils envoie un signal au pere\n");
```

```

        kill(pid_pere, SIGUSR1); /* demande de service */
    }

    exit(0);
} else { /* PERE */

    signal(SIGUSR1, Hand_Pere);

    while(1) {

        pause(); /* attend une demande de service */

        sleep(5); /* réalise le service */

    }

}
}

```

Fonctionnement de l'exemple :

- Le **fil**s envoie 10 signaux **SIGUSR1** au père.
- Le **père** utilise **pause()** pour attendre un signal et exécute **Hand_Pere** à chaque signal reçu.
- Permet de synchroniser les processus via les signaux de façon simple et asynchrone.

3.4. Exemples

1. Masquage temporaire d'un signal (**SIGINT**)

```

#include <stdio.h>

#include <signal.h>

#include <unistd.h>

int main() {

    signal(SIGINT, SIG_IGN);    // Ignorer SIGINT

    printf("Le signal SIGINT est ignoré \n");

    sleep(15);                  // Pendant 15 secondes

    printf("Rétablissement du signal \n");

    signal(SIGINT, SIG_DFL);    // Rétablir le traitement par défaut

    while(1);                   // Boucle infinie
}

```



```
}
```

- Le signal **SIGINT** est ignoré pendant 15 secondes, puis rétabli.

2. Protection par mot de passe avant interruption (**SIGINT**)

```
#include <stdio.h>

#include <stdlib.h>

#include <signal.h>

#include <unistd.h>

#include <string.h>

char PASS[12] = "informatique";

void password(int sig) {

    printf("Entrer le Password:\n");

    char entree[30];

    scanf("%s", entree);

    if (strcmp(entree, PASS) == 0) {

        signal(SIGINT, SIG_DFL);    // Rétablir SIGINT

        kill(getpid(), SIGINT);      // Envoyer SIGINT au processus

    } else {

        printf("Mot de passe incorrect\n");

    }

}

int main() {

    signal(SIGINT, password);        // Interceptor SIGINT

    while(1) {

        sleep(1);

        printf("le processus boucle.\n");

    }

}
```

```
}
```

- Le processus n'accepte **CTRL-C** qu'après validation du mot de passe.

Trace typique :

le processus boucle.

// Tapez CTRL-C

Entrer le Password:

informa

Mot de passe incorrect

le processus boucle.

// Tapez CTRL-C

Entrer le Password:

informatique

// Processus interrompu

3. Détection d'erreur arithmétique (**SIGFPE**)

```
#include <signal.h>
```

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
void hand_sigfpe() {
```

```
    printf("\nErreur division par 0 !\n");
```

```
    exit(1);
```

```
}
```

```
int main() {
```

```
    int a, b, resultat;
```

```
    signal(SIGFPE, hand_sigfpe);    // Handler pour erreur arithmétique
```

```
    printf("Taper a : "); scanf("%d", &a);
```

```
    printf("Taper b : "); scanf("%d", &b);
```

```
resultat = a / b;           // Déclenche SIGFPE si b=0

printf("La division de a par b = %d\n", resultat);

}
```

- Si division par zéro, le handler `hand_sigfpe()` affiche le message et termine le programme.

Trace typique :

Taper a: 12 Taper b: 0 Erreur division par 0 !

MAGUEMOUN SAMY